NMIMS MPSTME

B.Tech Computer Engineering

# DIGITAL IMAGE WATERMARKING

*A Hybrid DWT–DCT–SVD Approach for Robust, Imperceptible Copyright Protection*

Extended evaluation on 100 images, technology comparison, literature review and exploratory video test

*(Classical signal-processing method — no machine learning / deep learning)*

Subject: Image and Video Processing (IVP)

Submitted by

**Krishang Vikram Singh**

Roll No. B106 | Batch C1

First submission: July 2026 · Revised and extended edition: October 2026

# Abstract

This report presents the design, implementation and evaluation of a digital image watermarking system built entirely from classical signal-processing techniques — the Discrete Wavelet Transform (DWT), Discrete Cosine Transform (DCT) and Singular Value Decomposition (SVD). A 64×64 binary logo is embedded into the singular values of the DCT of the LL sub-band of a grey-scale host image. The first submission evaluated two images; this extended edition evaluates **100 unique images (94 real, 6 labelled synthetic)** under fourteen attacks plus an unattacked reference (JPEG at three qualities, Gaussian noise at two levels, salt-and-pepper noise, median, blur, sharpening, rotation at two angles, cropping, scaling, histogram equalisation), with fully seeded, logged and validated batch processing (3000 attack trials, 4000 α-sweep rows, 1600 control rows, 0 failures).

Imperceptibility is good (mean PSNR 36.0 dB, mean SSIM 0.992); the normalised correlation of the extracted logo reaches 0.75 in 85.1 % of the attack trials, with ≥ 95 % pass rates for JPEG (Q ≥ 20), median filtering, sharpening, scaling, blur and mild noise, 72 % for 15° rotation, 44 % for histogram equalisation and 1 % for border cropping. Three findings qualify the first submission: the α trade-off in the earlier figure was an artefact for JPEG — a zoomed NC axis (0.93–0.99) plus a truncation bias in the JPEG simulation that lowered NC only at α = 0.02 (the trade-off is real for noise, blur and rotation); the "block DCT" and "semi-blind" descriptions were wrong (global DCT; non-blind); and **a high normalised correlation does not prove ownership** — a different logo decoded with the owner's key scores NC 0.975, and unrelated images exceed 0.75 in 10–13 % of controls (the false-positive problem of SVD watermarking).

Against existing technology, the scheme is far more robust than LSB and 8×8 block-DCT embedding at equal PSNR but is not better than the open-source `blind-watermark` package, which has lower bit-error rates under most attacks at higher PSNR, is 13× slower to embed and is weaker under rotation. An exploratory per-frame test on two real video clips re-encoded with H.264 and MJPEG recovered the logo in all 720 decoded frames (NC ≥ 0.97). Every number in this report is generated from the recorded experiment outputs; limitations and unverified items are stated explicitly.

**Keywords:** digital watermarking, DWT, DCT, SVD, robustness, false-positive problem, PSNR, SSIM, normalised correlation.

# Table of Contents

**Abstract** ..... **2**
**1. Introduction** ..... **8**
1.1 Background and motivation ..... 8
1.2 Problem statement ..... 8
1.3 Objectives ..... 8
1.4 Scope and boundaries ..... 8
1.5 Applications ..... 9
1.6 Expected outcomes and main contributions ..... 9
**2. Literature Review** ..... **9**
2.1 Development of the techniques ..... 9
2.2 The false-positive problem ..... 10
2.3 Geometric attacks, video and learned approaches ..... 10
2.4 Paper-to-implementation mapping ..... 10
2.5 Research gaps and trade-offs relevant to this project ..... 10
**3. Existing Technology and Problem Analysis** ..... **11**
3.1 Existing tools and approaches ..... 11
3.2 Feature comparison ..... 11
3.3 Limitations relevant to the project ..... 11
3.4 Justification for the chosen methods ..... 11
**4. System Requirements and Architecture** ..... **12**
4.1 Functional requirements ..... 12
4.2 Non-functional requirements ..... 12
4.3 Hardware and software requirements ..... 12
4.4 System architecture and data flow ..... 12
4.5 Module description ..... 13
4.6 Input and output specification ..... 13
**5. Methodology and Algorithms** ..... **14**
5.1 Dataset preparation and preprocessing ..... 14
5.2 Mathematical formulation of the scheme ..... 14
5.3 Pseudocode ..... 14
5.4 Attack suite ..... 15
5.5 Evaluation metrics ..... 15
5.6 Video workflow (exploratory) ..... 15
5.7 Parameter selection and implementation decisions ..... 15
5.8 Summary of implemented transformations ..... 16
**6. Implementation** ..... **16**
6.1 Software structure ..... 16
6.2 Key functions and code excerpt ..... 16
6.3 Library usage ..... 16
6.4 Execution workflow and error handling ..... 16
6.5 Design decisions ..... 17
6.6 Tests ..... 17

**7. Experimental Setup and Evaluation..... 17**
7.1 The 100-image dataset..... 17
7.2 Hardware and software environment..... 19
7.3 Experimental procedure..... 19
7.4 Evaluation metrics and their limits..... 19
7.5 Statistical methods..... 19
7.6 Benchmarking methodology..... 19
7.7 Reproducibility..... 20
**8. Results and Discussion..... 20**
8.1 Imperceptibility (E1)..... 20
8.2 Robustness under attack (E2)..... 22
8.3 Protocol check..... 23
8.4 Which images fail?..... 24
8.5 Embedding strength α (E3)..... 25
8.6 False-positive controls (E4)..... 25
8.7 Failure cases..... 27
8.8 Comparison with existing technologies (S1, S2)..... 27
8.9 Exploratory video test (V1)..... 29
8.10 Comparison with the claims of the first submission..... 29
**9. Limitations and Future Scope..... 30**
9.1 Current limitations..... 30
9.2 Dataset limitations..... 30
9.3 Performance, quality and comparison limits..... 30
9.4 Future scope..... 30
**10. Conclusion..... 31**
**References..... 31**
**Appendix A: Installation and execution..... 33**
**Appendix B: Dataset sources and attribution..... 33**
**Appendix C: Supplementary results..... 33**
**Appendix D: Test results and configuration..... 35**
**Appendix E: Reproduction of the first submission..... 36**
**Appendix F: Corrections to the first report and presentation..... 36**
**Appendix G: Known limitations of this submission..... 37**

# List of Figures

**Figure 4.1: Watermark embedding pipeline (global 2-D DCT of the LL sub-band; the first submission's "block DCT" label was wrong).....13**
**Figure 4.2: Watermark extraction pipeline (non-blind, key-based).....13**
**Figure 7.1: All 100 images (standardised grey, 512×512), labelled with their IDs and source keys.....18**
**Figure 8.1: Distribution of PSNR and SSIM of the watermarked images (blue: real, orange: synthetic).....20**
**Figure 8.2: Original, watermarked and amplified (×10) absolute difference for four typical images (image_066, image_064, image_001, image_051) and two failure cases (image_055: near-black sky; image_100: checkerboard).....21**
**Figure 8.3: NC per attack over 100 images (box = interquartile range, line = median) and the share of images with NC ≥ 0.75..... 22**
**Figure 8.4: Detection pass rate per attack with 95 % Wilson intervals.....23**
**Figure 8.5: Extracted logo under every attack for one image (image_066, the cameraman).....23**
**Figure 8.6: NC after three attacks against image contrast (blue: real, orange: synthetic).....24**
**Figure 8.7: Mean NC by image category and attack family (real images; number of images per category in brackets)..... 24**
**Figure 8.8: α sweep over 100 images: PSNR (left), mean NC (middle) and pass rate (right) after four attacks; the dashed line in the middle panel marks NC = 0.75.....25**
**Figure 8.9: Mean NC for the genuine pair, for a different logo decoded with the owner's key (orange), and for unrelated images..... 26**
**Figure 8.10: Distribution of NC for genuine extraction and for unrelated-image controls.....26**
**Figure 8.11: Mean NC and mean ZNCC per attack..... 27**
**Figure 8.12: Median time per call for equivalent operations in different libraries (log scale; blue = implementation used by the project; orange includes process start-up and file I/O).....28**
**Figure 8.13: Mean BER of four schemes under nine attacks (PSNR in the legend).....29**
**Figure C.1: Timing of embedding and extraction (single thread, 4 parallel workers).....35**
**Figure C.2: Watermarked outputs of all 100 images (same order as Figure 7.1).....35**

# List of Tables

**Table 2.1: Mapping of literature to the implementation.....10**
**Table 3.1: Capability matrix (documentation-based; not benchmarked).....11**
**Table 4.1: Functional requirements.....12**
**Table 4.2: Software modules (source_code/src).....13**
**Table 5.1: The attack suite: 14 attacks plus the unattacked reference "No Attack" (15 conditions; identical to the first submission, now seeded). The first report called all 15 "attacks".....15**
**Table 5.2: Transformation summary (details, parameters, limitations and complexity: documentation/TRANSFORMATIONS_REFERENCE.md).....16**
**Table 7.1: Composition of the dataset by category.....18**
**Table 7.2: Experiments.....19**
**Table 8.1: Robustness over 100 images per attack (uint8 protocol). Total: 1277 of 1500 trials ≥ 0.75 (85.1 %); mean NC 0.843.....22**
**Table 8.2: Scheme comparison on 30 real images with the same 4095-bit payload (BER 0.5 = chance).....28**
**Table 8.3: Per-frame embedding on 60 frames of each of two clips (vtest.avi, Megamind.avi), re-encoded with FFmpeg; means over the two clips.....29**
**Table 8.4: Claims of the first submission against the 100-image evaluation.....29**
**Table B.1: Real-image sources (the repository licence is not a statement about each photograph).....33**
**Table C.1: Mean NC by category and attack family (all 100 images, uint8 protocol).....33**
**Table C.2: α sweep (excerpt of results/tables/alpha_sweep_summary.csv).....34**
**Table C.3: False-positive controls (E4).....34**
**Table E.1: Re-execution of the original two-image experiment against Table 5.1 of the first report. Differences are due to seeded noise and JPEG rounding.....36**

# Abbreviations and symbols

*Table 0.1: Abbreviations and symbols.*

| Symbol | Meaning |
|---|---|
| DWT / DCT / SVD | discrete wavelet transform / discrete cosine transform / singular value decomposition |
| LL | approximation (low-pass) sub-band of the DWT |
| α | embedding strength (0.10 unless stated) |
| S, S_w, S_orig, S_att | singular values of the host sub-band, of the logo, of the original host, of the attacked image |
| U, V, U_w, V_w | singular-vector matrices of host and logo |
| PSNR / SSIM | peak signal-to-noise ratio / structural similarity index |
| NC / ZNCC / BER | normalised correlation / zero-mean NC / bit error rate |
| LSB | least significant bit |
| CRF / MJPEG / FFV1 | constant-rate-factor of x264 / motion JPEG / lossless codec |
| FPP | false-positive problem |

# 1. Introduction

## 1.1 Background and motivation

Digital images and videos are trivially copied, redistributed and repurposed. Photographers, medical institutions, publishers and manufacturers therefore need a way of embedding an invisible, verifiable ownership signature in their visual content without degrading it. Digital watermarking hides auxiliary information directly in the pixel or transform-domain data of the host media [17][32].

Two broad families exist. Spatial-domain methods, such as least-significant-bit (LSB) substitution [10], are simple but fragile. Transform-domain methods (DCT, DWT, SVD and their hybrids) spread the watermark over frequency components and are considerably more robust to compression and everyday signal processing [1][2][3]. This project implements a hybrid DWT–DCT–SVD scheme and evaluates it; the present report extends the first submission (two test images) with a 100-image evaluation, controlled experiments, a comparison with existing technology and a verified literature review.

## 1.2 Problem statement

Design an invisible digital image watermarking system that (a) introduces minimal, imperceptible distortion, (b) survives common real-world distortions such as compression, noise, filtering and geometric change, and (c) is fully explainable through closed-form linear-algebraic operations, with no learned component — and **establish, with measurements, how far each of these properties actually holds**.

## 1.3 Objectives

1. Implement a hybrid DWT–DCT–SVD embedding and extraction pipeline from first principles (original objective).
2. Quantify imperceptibility (PSNR, SSIM) and robustness (NC, BER, and the added ZNCC) under a suite of 14 attacks (plus an unattacked reference) on **100 unique images** (extended objective).
3. Characterise the imperceptibility–robustness trade-off as a function of the embedding strength $\alpha$.
4. Identify and explain failure modes, including whether a high NC really proves that *this* watermark is present (control experiments).
5. Compare the implementation with established libraries and with an existing open-source watermarking package, using controlled benchmarks.
6. Test, as an exploratory extension, whether the scheme can be applied frame by frame to real video that is re-encoded with lossy codecs.

## 1.4 Scope and boundaries

- 8-bit **grey-scale** images; colour is handled only by converting to luminance (ITU-R BT.601) during dataset preparation.
- No machine learning or deep learning at any stage (design constraint of the project); learned watermarking is discussed only as context.
- The scheme is **non-blind**: extraction needs a key that contains the original singular values.

- Video is an **exploratory** test (60 frames from each of two clips); temporal attacks, colour video and hardware codecs are out of scope.
- Only operations present in the project's code are documented as transformations; edge detection, CLAHE, morphology, perspective warps etc. are not part of this project.

### 1.5 Applications

Ownership marking of photographs, medical-image provenance, tracing of unauthorised redistribution of publications, and certification of technical imagery. The measurements in Chapter 8 show which of these uses the scheme can support (robust against re-compression and filtering) and which it cannot (geometric cropping; proof of ownership without additional safeguards).

### 1.6 Expected outcomes and main contributions

- A reproducible, tested implementation (43 unit/integration tests, seeded experiments, config-driven batch runner).
- A category-stratified evaluation set of 100 unique images (94 real, 6 clearly labelled synthetic) and 3000 attack trials (+4000 sweep and 1600 control rows) with full status logging.
- **A measured false-positive problem:** a different logo decoded with the owner's key still yields NC 0.975 (genuine 0.977), so NC alone cannot prove ownership.
- Corrections to the first submission (global rather than block DCT, non-blind rather than semi-blind, an α-trade-off figure that exaggerated a 0.05 change, unreproducible noise attacks) — Appendix F.
- A controlled comparison with PyWavelets/SciPy/NumPy/OpenCV/Pillow/scikit-image/ImageMagick implementations and with the `blind-watermark` package, showing where the project is and is not competitive.

## 2. Literature Review

This chapter summarises the literature relevant to the techniques the project actually uses. The first submission cited Cox et al. [1], Liu and Tan [2], Ganic and Eskicioglu [3], Gonzalez and Woods [31] and library documentation; it did not state that code was derived from them, so they are treated here as *foundational literature identified during this review*. Bibliographic details were checked as recorded in the reference list (status flags in `research/REFERENCES.md`).

### 2.1 Development of the techniques

**Spatial domain.** Van Schyndel, Tirkel and Osborne [10] are the usual starting point for digital image watermarking in the spatial domain. LSB embedding offers high capacity and almost no distortion but no robustness; in our own baseline (Chapter 8) a 4095-bit LSB payload reaches 69.5 dB PSNR yet has a bit-error rate of 0.43 after JPEG Q=50 and 0.50 after Gaussian noise (chance level is 0.5).

**Spread spectrum in a transform domain.** Cox, Kilian, Leighton and Shamoon [1] argued that a watermark must be placed in the perceptually significant spectral components if it is to survive processing; a pseudo-random Gaussian sequence is inserted into the most significant spectral components and detected by correlation. This established the imperceptibility–robustness trade-off studied here through α. Block-DCT methods such as Koch and Zhao [11] tie the embedding to the JPEG model; surveys are given by Hartung and Kutter [17] and Cox *et al.* [32].

**Wavelets.** Mallat's multiresolution theory [12] gives the DWT used here. Embedding in the approximation (LL) band trades visibility for robustness; embedding in detail bands does the opposite.

**Singular value decomposition.** Liu and Tan [2] introduced SVD-based ownership watermarking; Ganic and Eskicioglu [3] combined DWT and SVD and embedded in all sub-bands; Sverdlov, Dexter and Eskicioglu [6] did the same with the DCT; Lai and Tsai [7] proposed a DWT-SVD scheme. Hybrids that cascade DWT, DCT and SVD appear in later work, for example Bekkouch and Faraoun [8] (reversible, two watermarks) and Begum, Ferdush and Uddin [9] (blind, Arnold-map scrambling). These authors report better robustness or imperceptibility than earlier schemes; **we did not re-evaluate those claims**.

## 2.2 The false-positive problem

Zhang and Li [4] published a comment on Liu and Tan [2], and Ting [5] studied ambiguity attacks on the Ganic–Eskicioglu scheme [3]. Review literature found during the search describes the common cause: if the watermark's singular values are embedded and its singular vectors are kept as the key, the detector reconstructs the key owner's watermark regardless of what was embedded. This report contributes a direct measurement of the effect for this implementation (Section 8.6).

## 2.3 Geometric attacks, video and learned approaches

Lin *et al.* [22] explain that rotation, scale and translation defeat correlation detectors unless the detection domain is made invariant (log-polar Fourier representation); our rotation and cropping results are consistent with that. Hartung and Girod [23] treat watermarking of uncompressed and compressed video. Learned watermarking (HiDDeN [29], StegaStamp [30]) trains encoder/decoder networks against a distribution of distortions; the project excludes it by design, and no experiment here compares against it.

## 2.4 Paper-to-implementation mapping

*Table 2.1: Mapping of literature to the implementation.*

| Ref. | Technique | Project function / experiment | Relationship |
|---|---|---|---|
| [1] Cox et al. | embedding in significant coefficients; α trade-off | `embed_watermark` (α); E3 | conceptual foundation |
| [2] Liu & Tan | singular-value embedding | `embed_watermark` step 5 | closest ancestor (the project adds the logo's singular values) |
| [3] Ganic & Eskicioglu | DWT-SVD hybrid | `embed_watermark` (DWT + SVD) | conceptual foundation |
| [4][5] | false-positive / ambiguity critique | experiment E4 | motivates the controls |
| [10][11] | LSB; block-DCT | baselines in `benchmark_tech.py` | comparison |
| [12] Mallat | DWT theory | `pywt.dwt2` | theory |
| [13][14] | DCT; JPEG | `dct2`; `jpeg_compression` | theory / attack |
| [16] Wang et al. | SSIM | `metrics.ssim` | metric |
| [22] Lin et al. | RST-resilient detection | rotation, cropping, scaling attacks | explains geometric fragility |
| [23] Hartung & Girod | video watermarking | `video_demo.py` | related work |
| [28] blind-watermark | block DWT-DCT-SVD, blind | `BlindWMScheme` | measured comparison |

## 2.5 Research gaps and trade-offs relevant to this project

1. **Identity binding:** the scheme does not bind the logo to the image (Section 8.6).
2. **Geometry:** global SVD embedding is geometry-fragile; block-based or invariant-domain embedding [22] is the standard remedy.

3. **Metric reliability:** plain NC is dominated by the logo's first singular value — a blank white image already scores NC = 0.359 and a rank-1 approximation 0.868 for this logo.
4. **Evaluation breadth:** hybrid schemes in [7]–[9] are, to our reading of their abstracts, evaluated on small standard-image sets; a 100-image stratified evaluation is a contribution of this work.

*The text of this chapter was written from verified titles, abstracts and reference-list entries; where only secondary descriptions were available this is stated.*

# 3. Existing Technology and Problem Analysis

## 3.1 Existing tools and approaches

The project is an algorithm built on libraries; it is not a general-purpose imaging toolkit. Relevant technologies are NumPy/SciPy (SVD, DCT, filters), PyWavelets (DWT), scikit-image (geometry, metrics), Pillow (JPEG), OpenCV (DCT, SVD, filters, video I/O), ImageMagick (command-line conversion), FFmpeg (video codecs), and at scheme level the `blind-watermark` package [28]. Machine-learning watermarkers [29][30] are out of scope.

## 3.2 Feature comparison

*Table 3.1: Capability matrix (documentation-based; not benchmarked).*

| Capability | NumPy | SciPy | PyWavelets | scikit-image | Pillow | OpenCV | ImageMagick | FFmpeg |
|---|---|---|---|---|---|---|---|---|
| 2-D DCT | – | ✓ | – | – | – | ✓ | – | – |
| SVD | ✓ | ✓ | – | – | – | ✓ | – | – |
| DWT | – | – | ✓ | (via PyWavelets) | – | – | – | – |
| JPEG codec | – | – | – | (I/O plug-ins) | ✓ | ✓ | ✓ | MJPEG |
| filters, rotate, resize | – | ✓ | – | ✓ | ✓ | ✓ | ✓ | ✓ |
| PSNR / SSIM | – | – | – | ✓ / ✓ | – | PSNR (SSIM: contrib) | `compare` | – |
| video decode/encode | – | – | – | – | – | ✓ | – | ✓ |
| licence (metadata) | BSD-3 | BSD | MIT/BSD-3 | BSD | MIT-CMU | Apache-2.0 | IM licence | LGPL/GPL |

## 3.3 Limitations relevant to the project

No single library covers DWT + DCT + SVD + JPEG + metrics, so a combination is necessary; PyWavelets supplies the DWT that OpenCV lacks. An existing blind watermarking package exists [28] and outperforms the project in several attacks (Chapter 8), so novelty cannot be claimed on robustness. The first submission's central design argument — hybrids beat single transforms — was taken from the literature and is not independently tested here.

## 3.4 Justification for the chosen methods

- The singular-value scheme is closed-form and explainable, in line with the project's constraint of no learned components.
- PyWavelets + SciPy + NumPy + Pillow + scikit-image give numerically agreeing results (≤ 1.5e-16 relative for DCT/SVD; Section 8.8) and the whole 100-image experiment runs in about 80 s on four cores, so a change of library is not justified on performance grounds.

- The Haar wavelet and global DCT were retained from the first submission to preserve its functionality and allow direct comparison.

# 4. System Requirements and Architecture

## 4.1 Functional requirements

*Table 4.1: Functional requirements.*

| ID | Requirement | Where satisfied |
|---|---|---|
| FR1 | Load any common image file safely (colour, alpha, 16-bit, odd sizes) and standardise it | `image_io.load_gray_square` |
| FR2 | Embed a binary logo into a grey image with a single strength parameter α | `embed_watermark` |
| FR3 | Extract the logo from a possibly attacked image using the key | `extract_watermark` |
| FR4 | Apply a reproducible suite of 14 attacks plus an unattacked reference | `attacks.make_attack_suite` |
| FR5 | Compute PSNR, SSIM, NC, ZNCC, BER | `metrics.py` |
| FR6 | Process 100 images in batch, record status, time and parameters per operation, validate outputs | `run_batch_experiment.py` |
| FR7 | Produce tables and figures from recorded results only | `analysis.py` |
| FR8 | Apply the scheme to video frames (exploratory) | `video_demo.py` |

## 4.2 Non-functional requirements

- **Reproducibility:** all stochastic steps seeded; two complete runs gave identical metrics (maximum absolute difference 0.0).
- **Robustness of the tooling:** a failing image or attack is recorded, not fatal; corrupt, empty and unsupported files raise `ImageLoadError`.
- **Performance:** a 512×512 image is embedded in about 37 ms and extracted in about 20 ms on one CPU core; peak resident memory of a worker is 166 MB.
- **Portability:** pure Python with NumPy/SciPy stack; no GPU required.

## 4.3 Hardware and software requirements

Tested on a 4-vCPU Intel Xeon @ 2.1 GHz with 15 GB RAM, Linux, Python 3.13.16, NumPy 2.5.3, SciPy 1.18.1, scikit-image 0.26.0, Pillow 12.3.0, Pandas 3.0.5. Optional: OpenCV 5.0.0 and FFmpeg 6.1.1 (video test), ImageMagick 6.9.12 (one benchmark row), `blind-watermark` 0.4.4 (comparison).

## 4.4 System architecture and data flow

Two pipelines share one key: an embedding pipeline run once by the owner and an extraction pipeline run whenever ownership is to be verified (Figures 4.1 and 4.2). In the batch experiment the watermarked image is rounded to 8 bit, attacked, and then passed to the extractor.

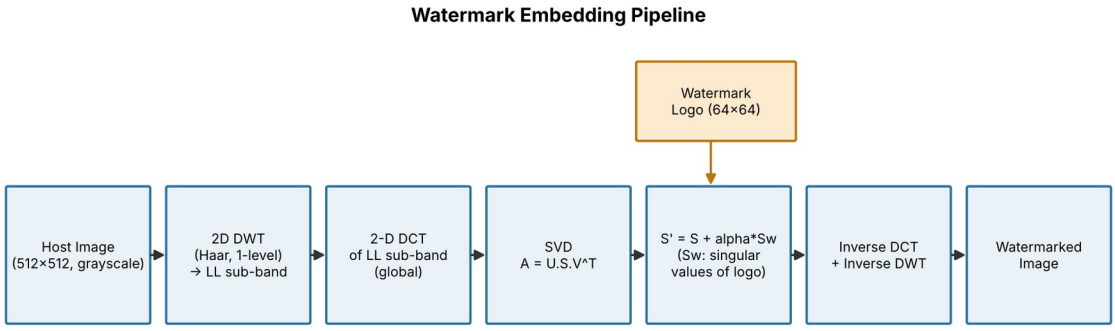



*Figure 4.1: Watermark embedding pipeline (global 2-D DCT of the LL sub-band; the first submission's "block DCT" label was wrong).*

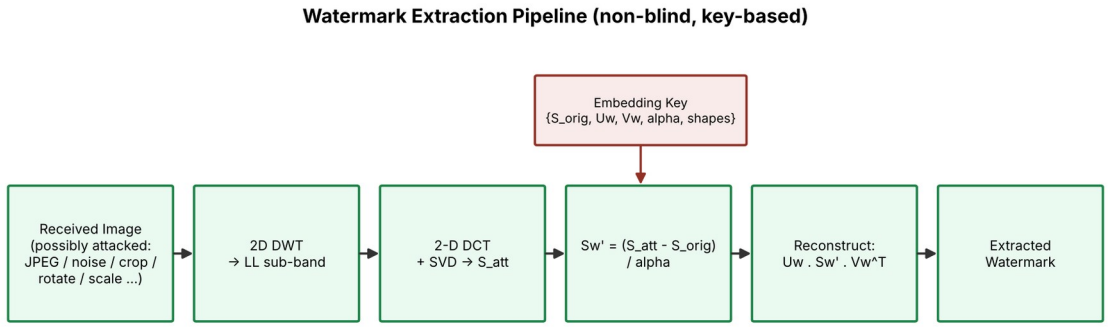



*Figure 4.2: Watermark extraction pipeline (non-blind, key-based).*

## 4.5 Module description

*Table 4.2: Software modules (source_code/src).*

| Module | Responsibility |
|---|---|
| `watermark_core.py` | DWT + DCT + SVD embedding / extraction (numerics unchanged from the first submission, validation added) |
| `metrics.py` | MSE, PSNR, SSIM, NC, ZNCC, BER with shape checks |
| `attacks.py` | 14 attacks + reference; `make_attack_suite(seed)` |
| `logo.py` | procedural 64×64 binary logo |
| `image_io.py` | safe decoding, standardisation, PNG verification |
| `dataset_manifest.py`, `build_dataset.py` | sources, licences, near-duplicate rejection, synthetic controls, `metadata.csv` |
| `run_batch_experiment.py` | E1–E4 with multiprocessing, per-operation status |
| `analysis.py`, `analysis_tech.py` | statistics, tables, figures |
| `benchmark_tech.py` | component and scheme benchmarks |
| `video_demo.py` | exploratory video test |
| `run_experiment.py`, `make_diagram.py` | legacy two-image experiment, architecture diagrams |

## 4.6 Input and output specification

- **Input:** any decodable image (PNG/JPEG/...); standardised to a 512×512 8-bit grey array. Watermark: 2-D array, any size (64×64 binary logo used).

- **Output of embedding:** watermarked 512×512 float array in [0,255] (stored as 8-bit PNG) and a key holding the original singular values (≈ 2.1 MB per image including the unused `U`, $V^t$).
- **Output of extraction:** a 64×64 grey image; NC/ZNCC/BER against the logo.

# 5. Methodology and Algorithms

## 5.1 Dataset preparation and preprocessing

Every source image is decoded with Pillow, alpha is composited over white, 16-bit data is rescaled linearly, colour is converted to grey with the BT.601 luma `L = 0.299R + 0.587G + 0.114B`, the largest centred square is cropped (no aspect distortion), and the result is resampled to 512×512 with Lanczos. The 100-image set and its provenance are described in Chapter 7.

## 5.2 Mathematical formulation of the scheme

**Haar DWT.** A one-level 2-D Haar transform splits the host H into four sub-bands; for each 2×2 block (a, b; c, d) the approximation is

$$LL = (a + b + c + d) / 2 \tag{1}$$

and the three detail bands are the corresponding sums and differences. The watermark is embedded in LL (256×256).

**2-D DCT (global).** The orthonormal DCT-II is applied to the **whole** LL band:

$$F(u,v) = \alpha(u)\,\alpha(v)\,\Sigma_x\,\Sigma_\gamma\, f(x,y)\cos[(2x+1)u\pi / 2M]\cos[(2y+1)v\pi / 2N] \tag{2}$$

with $\alpha(0) = \sqrt{(1/M)}$ and $\alpha(u>0) = \sqrt{(2/M)}$. **SVD.** The DCT matrix and the (resized) logo are factorised:

$$A = U \cdot \mathrm{diag}(S) \cdot V^T, \qquad W = U_w \cdot \mathrm{diag}(S_w) \cdot V_w^T \tag{3}$$

**Embedding rule.** With embedding strength α (0.10 in the main experiment):

$$S' = S + \alpha \cdot S_w, \qquad A' = U \cdot \mathrm{diag}(S') \cdot V^T \tag{4}$$

followed by the inverse DCT, the inverse DWT and clipping to [0,255]. **Extraction rule.** From the received image the same DWT–DCT–SVD gives S_att, and

$$S_w' = (S_att - S_orig) / \alpha, \qquad W' = U_w \cdot \mathrm{diag}(S_w') \cdot V_w^T \tag{5}$$

The key therefore contains S_orig, U_w, $V_w^T$, α and the shapes; U and $V^T$ of the host are stored but not used by the extractor. Because S_orig is derived from the unmarked host the scheme is **non-blind**, and because the logo's singular vectors are in the key the extractor outputs the key owner's logo whatever singular values it finds (Section 8.6).

## 5.3 Pseudocode

```
EMBED(host, logo, alpha):
    LL, details = haar_dwt2(host)
    A  = dct2(LL)                       # global 256x256
    U, S, Vt   = svd(A)
    Wr = resize(logo, shape(LL))
    Uw, Sw, Vwt = svd(Wr)
    S' = S + alpha * Sw
    LL' = idct2(U @ diag(S') @ Vt)
    out = clip(inverse_haar(LL', details), 0, 255)
    key = {S, Uw, Vwt, alpha, shapes}
    return out, key
```

```
EXTRACT(image, key):
    LL, _ = haar_dwt2(image);  if shape(LL) != key.shape: LL = resize(LL, key.shape)
    S_att = svd_values(dct2(LL))
    Sw'   = (S_att - key.S) / key.alpha
    W'    = key.Uw @ diag(Sw') @ key.Vwt
    return clip(resize(W', logo_shape), 0, 255)
```

## 5.4 Attack suite

*Table 5.1: The attack suite: 14 attacks plus the unattacked reference "No Attack" (15 conditions; identical to the first submission, now seeded). The first report called all 15 "attacks".*

| Family | Attack (parameters) | Implementation |
|---|---|---|
| compression | JPEG quality 90, 50, 20 | Pillow, baseline JPEG, floats rounded to 8 bit first |
| noise | Gaussian $\sigma^2$=0.005 and 0.02 (on [0,1]; $\sigma \approx 18$, 36 grey levels); salt-and-pepper 2 % | scikit-image `random_noise`, seeded |
| filtering | median 3×3; Gaussian blur $\sigma$=1.5; unsharp mask ($\sigma$=1, amount 1) | SciPy `ndimage` |
| geometric | rotation 5° and 15° (centre, bilinear, edge replicate); border zeroing with 25 % of each dimension removed (**43.75 % of the area**); down/up-scaling 0.5× | scikit-image `rotate`, `resize`; NumPy |
| global intensity | histogram equalisation | scikit-image `equalize_hist` |

## 5.5 Evaluation metrics

$$\mathrm{PSNR} = 10\log_{10}(255^2/\mathrm{MSE}), \quad \mathrm{NC} = \Sigma\, W\cdot W' / (\|W\|\,\|W'\|), \quad \mathrm{ZNCC} = \mathrm{NC\ of\ the\ mean\text{-}centred\ images} \tag{6}$$

SSIM [16] is used with scikit-image's defaults (7×7 window). BER is the fraction of logo pixels that differ after thresholding at 127.5. **ZNCC is new:** plain NC is not mean-centred, so any bright image already scores NC = 0.359 against this logo (the square root of its 12.9 % white fraction); ZNCC removes that offset. The 0.75 recognisability threshold is a convention of the project that we keep for comparability; we could not trace it to a verified literature source.

## 5.6 Video workflow (exploratory)

Frames are read with OpenCV, standardised like still images, watermarked individually (each frame has its own key because the scheme is non-blind), encoded by FFmpeg (FFV1 lossless, H.264 at CRF 18/23/28, MJPEG at q 5/15), decoded, and extracted frame by frame. Temporal redundancy is not exploited.

## 5.7 Parameter selection and implementation decisions

- **$\alpha$ = 0.10** is kept from the first submission. It gives about 36 dB PSNR and, at that value, 100 % pass rate under JPEG Q=50; Section 8.5 shows how the pass rate under other attacks changes with $\alpha$.
- **Protocol:** the watermarked image is rounded to 8 bit before attacking (primary), because that is what a saved file contains; the first submission's float protocol is reported as a check.
- **Seeds:** attacks use `20260713 + 100·index (+ offset)`; control images are drawn with the same base seed.

### 5.8 Summary of implemented transformations

*Table 5.2: Transformation summary (details, parameters, limitations and complexity: documentation/TRANSFORMATIONS_REFERENCE.md).*

| # | Transformation | Function | Library |
|---|---|---|---|
| T1 | grey conversion, crop, Lanczos resize | `load_gray_square` | Pillow |
| T2 | 1-level Haar DWT and inverse | `embed/extract_watermark` | PyWavelets |
| T3 | 2-D DCT-II and inverse (global) | `dct2`, `idct2` | SciPy fftpack |
| T4 | SVD of host sub-band and logo | `embed_watermark` | NumPy |
| T5 | singular-value embedding / extraction | `embed/extract_watermark` | NumPy |
| T6 | anti-aliased resize (logo ↔ sub-band) | inside T5 | scikit-image |
| T7 | procedural binary logo | `make_watermark_logo` | Pillow |
| T8–T17 | JPEG, Gaussian & salt-and-pepper noise, median, blur, rotation, border cropping, scaling, histogram equalisation, sharpening | `attacks.py` | Pillow, scikit-image, SciPy |
| T18–T19 | PSNR, SSIM, MSE, NC, ZNCC, BER | `metrics.py` | scikit-image, NumPy |
| T20 | frame extraction, per-frame embedding, codec round-trip | `video_demo.py` | OpenCV, FFmpeg |

# 6. Implementation

## 6.1 Software structure

The repository keeps the first submission's four modules and adds loaders, a dataset builder, a batch runner, analysis and benchmark scripts (Table 4.2). Original files are preserved in `source_code/original_backup/`; every change is listed in `documentation/CHANGELOG.md`.

## 6.2 Key functions and code excerpt

The embedding core is unchanged from the first submission (excerpt of `watermark_core.py`):

```
LL, (LH, HL, HH) = pywt.dwt2(host, wavelet)
wm_resized = resize(watermark, target.shape, anti_aliasing=True, preserve_range=True)
target_dct = dct2(target)
U, S, Vt   = np.linalg.svd(target_dct, full_matrices=False)
Uw, Sw, Vtw = np.linalg.svd(wm_resized, full_matrices=False)
S_mod = S + alpha * Sw
target_mod = idct2((U * S_mod) @ Vt)
```

## 6.3 Library usage

NumPy (SVD, array algebra), SciPy (`fftpack.dct`, `ndimage` filters), PyWavelets (`dwt2`/`idwt2`), scikit-image (resize, rotate, noise, equalisation, PSNR/SSIM), Pillow (decoding, JPEG), Pandas/Matplotlib (tables, figures), OpenCV and FFmpeg (video test), python-docx/python-pptx (report and deck builders).

## 6.4 Execution workflow and error handling

`build_dataset.py` → `run_batch_experiment.py` → `analysis.py` (→ `benchmark_tech.py`, `video_demo.py`). Each attack/extraction is wrapped so that an exception is stored as `status = error` with text; image-level failures are logged and the run continues. Outputs are re-opened and verified (shape, dtype, finiteness). The run summary records 100/100 images, 3000/3000 attack trials and 4000/4000 sweep rows without failure.

### 6.5 Design decisions

- Numerical behaviour of the original embed/extract code was **not** modified, so the first submission's numbers remain reproducible (Appendix E).
- Input validation rejects non-2-D, tiny, NaN/Inf or non-numeric input and $\alpha \le 0$.
- A variant that embeds the logo's principal component was tried and **rejected**: because the key then contains the whole logo, an unwatermarked image still decodes at NC = 0.98 (`results/exp_principal_component_variant.txt`).
- Per-image parallelism with one BLAS thread per process keeps timings comparable.

### 6.6 Tests

43 tests (pytest) cover DCT round trips, embedding shape/range/key, clean recovery, monotone PSNR in α, odd sizes, resized input, invalid inputs, metric definitions and boundary values, attack seeding and the cropped area, image loading of RGB/RGBA/16-bit/bilevel/palette/corrupt/empty/unsupported files, and completeness of the dataset and results. All 43 pass (`results/test_results.txt`).

## 7. Experimental Setup and Evaluation

### 7.1 The 100-image dataset

The set contains 100 unique 512×512 grey images: 94 real images from open GitHub-hosted sample collections and 6 synthetic control images (gradient, radial gradient, grating, white noise, low-contrast noise, checkerboard) that are labelled as synthetic everywhere. Brightness classes: mid 87, dark 7, bright 6; contrast classes: medium 62, low 19, high 19. Near-duplicates were rejected by a 64-bit difference hash and uniqueness is asserted by a test. Sources, licences and exclusions are documented in `data/README.md`; per-image provenance is in `data/metadata.csv`.

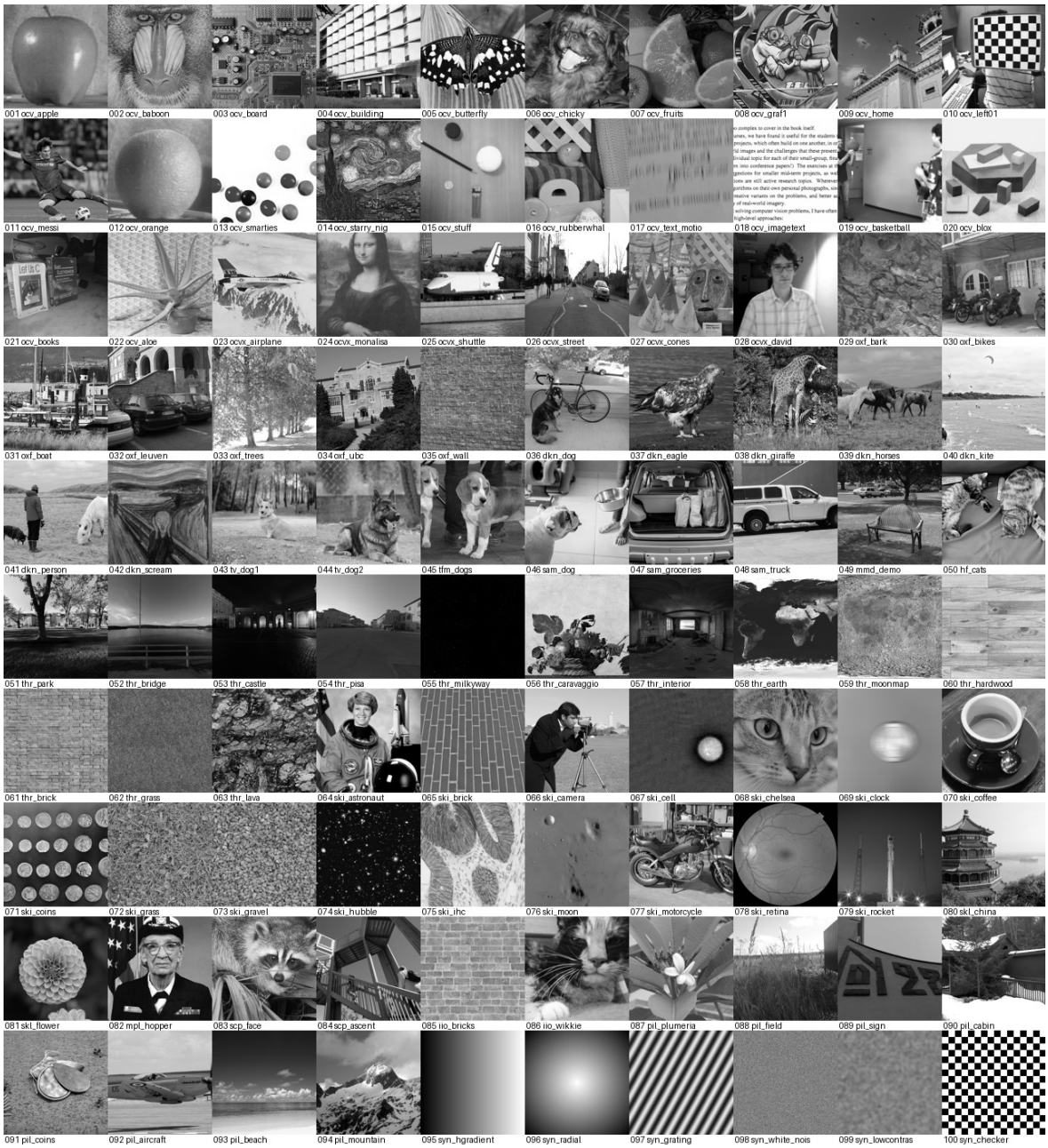



*Figure 7.1: All 100 images (standardised grey, 512×512), labelled with their IDs and source keys.*

*Table 7.1: Composition of the dataset by category.*

| category | images | category | images |
|---|---|---|---|
| animal | 14 | painting_art | 4 |
| urban_architecture | 13 | plant_object | 3 |
| object | 12 | scientific_medical | 3 |
| texture_natural | 10 | indoor_person | 3 |
| natural_scene | 10 | astronomy_space | 3 |
| synthetic_control | 6 | scientific_map | 2 |
| aerial_vehicle | 4 | document_text | 2 |
| person_portrait | 4 | person_sports | 2 |
| indoor_object | 4 | object_fine_detail | 1 |

**Provenance and rights.** Hosting-repository licences (Apache-2.0, BSD, MIT, HPND, "YOLO") are recorded per image, but the repository licence is not a statement about each photograph. Where the upstream project states a licence for an image (CC0 images from scikit-image, NASA public domain, CC

BY 2.0 images from scikit-learn) it is quoted in the metadata; for most others nothing is documented upstream, which is stated. The images are used only for non-commercial academic evaluation. Only `raw.githubusercontent.com` and PyPI were reachable when the set was built, which limits the choice of sources.

## 7.2 Hardware and software environment

4-vCPU Intel Xeon @ 2.1 GHz, 15 GB RAM, Linux 6.18, Python 3.13.16; NumPy 2.5.3, SciPy 1.18.1, scikit-image 0.26.0, Pillow 12.3.0, PyWavelets (module attribute 1.8.0, distribution metadata 1.10.0), OpenCV 5.0.0, FFmpeg 6.1.1, ImageMagick 6.9.12-98. No GPU.

## 7.3 Experimental procedure

*Table 7.2: Experiments.*

| ID | Experiment | Size | Recorded |
|---|---|---|---|
| E1 | embed at α = 0.10, save 8-bit PNG, re-open and verify | 100 images | PSNR, SSIM, differences, time, key size |
| E2 | 15 conditions (14 attacks + reference) × 2 protocols (uint8 primary, float check) | 3000 trials | NC, ZNCC, BER, times, status |
| E3 | α sweep (8 values) × {no attack, JPEG Q=50, Gaussian noise 0.02, blur 1.5, rotation 5°} | 4000 rows | PSNR, SSIM, NC, ZNCC, BER |
| E4 | false-positive controls (a–d, genuine reference) | 1600 rows | NC, ZNCC, BER |
| S1 | scheme comparison: project vs `blind-watermark` vs block-DCT vs LSB | 30 images × 9 attacks × 4 | PSNR, BER, NC, time |
| S2 | component benchmark (10 operations, several libraries) | 3 images, 9 repeats | median time, agreement |
| V1 | video: 2 clips × 60 frames × 6 codec settings | 720 decoded frames | PSNR, NC, BER, bit-rate, time |

## 7.4 Evaluation metrics and their limits

PSNR and SSIM compare the watermarked image with the *unwatermarked original* and are appropriate for imperceptibility. NC, ZNCC and BER compare the extracted logo with the original logo and are **conditional on the key**; they do not show that the logo was embedded (Section 8.6). A higher metric is not automatically better: PSNR is nearly host-independent here, SSIM can collapse on near-constant images, and edge-detection style metrics do not apply because the project has no edge-detection operation.

## 7.5 Statistical methods

Means with 95 % bootstrap confidence intervals (2000 resamples), pass rates with Wilson 95 % intervals, Spearman rank correlations (28 p-values, not corrected for multiplicity), and Mann–Whitney AUC for genuine versus control NC. The 94 real images are a convenience sample, so intervals describe sampling variability within this set, not all photographs.

## 7.6 Benchmarking methodology

Component benchmark: same 512×512 (or 256×256) input, one thread, in-memory (file I/O excluded), two warm-up calls, nine repeats, median over three randomly chosen real images; process-based tools (ImageMagick) are flagged. Scheme benchmark: identical 4095-bit payload (the last of the 4096 logo bits is dropped because `blind-watermark` needs strictly fewer bits than its 4096 blocks), identical attacks, 8-bit rounding, 30 random real images; the block-DCT baseline's strength (Δ = 48) was tuned on five other images so that its PSNR (35.4 dB) matches the project's (35.6 dB).

### 7.7 Reproducibility

Everything is seeded; two complete executions of the batch experiment produced identical metrics (maximum absolute difference 0.0 over 3000 + 4000 + 1600 + 100 rows; timings vary by 5–10 %). Commands are in Appendix A.

## 8. Results and Discussion

### 8.1 Imperceptibility (E1)

Across the 100 images at α = 0.10 the mean PSNR of the stored 8-bit watermarked image is **35.99 dB** (median 35.60, minimum 35.53, maximum 43.78; real images only: 35.92 dB) and the mean SSIM is **0.9918** (real only: 0.9929). No image falls below 30 dB; 3 images have SSIM < 0.98 (image_055, image_074, image_100). The mean absolute pixel change is 3.58 grey levels, and PSNR is almost independent of the host (82 of 100 images lie within 35.5–35.75 dB) because the added energy is set by the logo's singular values, not by the host.

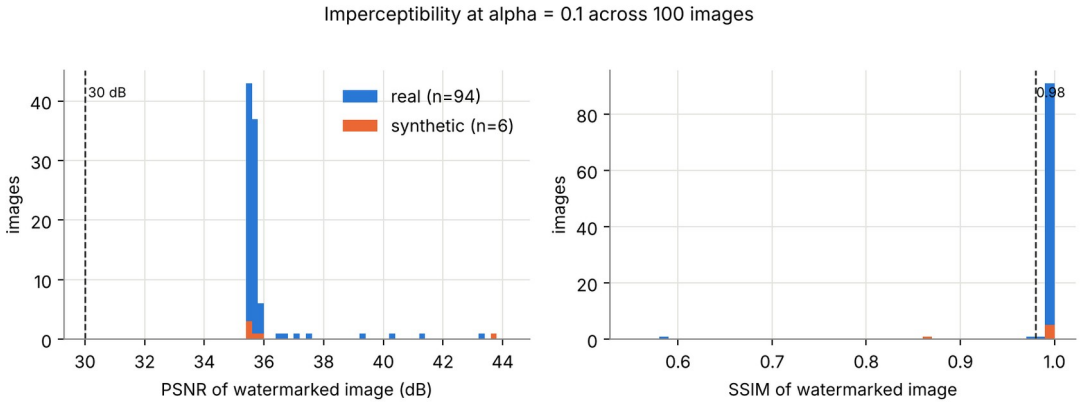



*Figure 8.1: Distribution of PSNR and SSIM of the watermarked images (blue: real, orange: synthetic).*

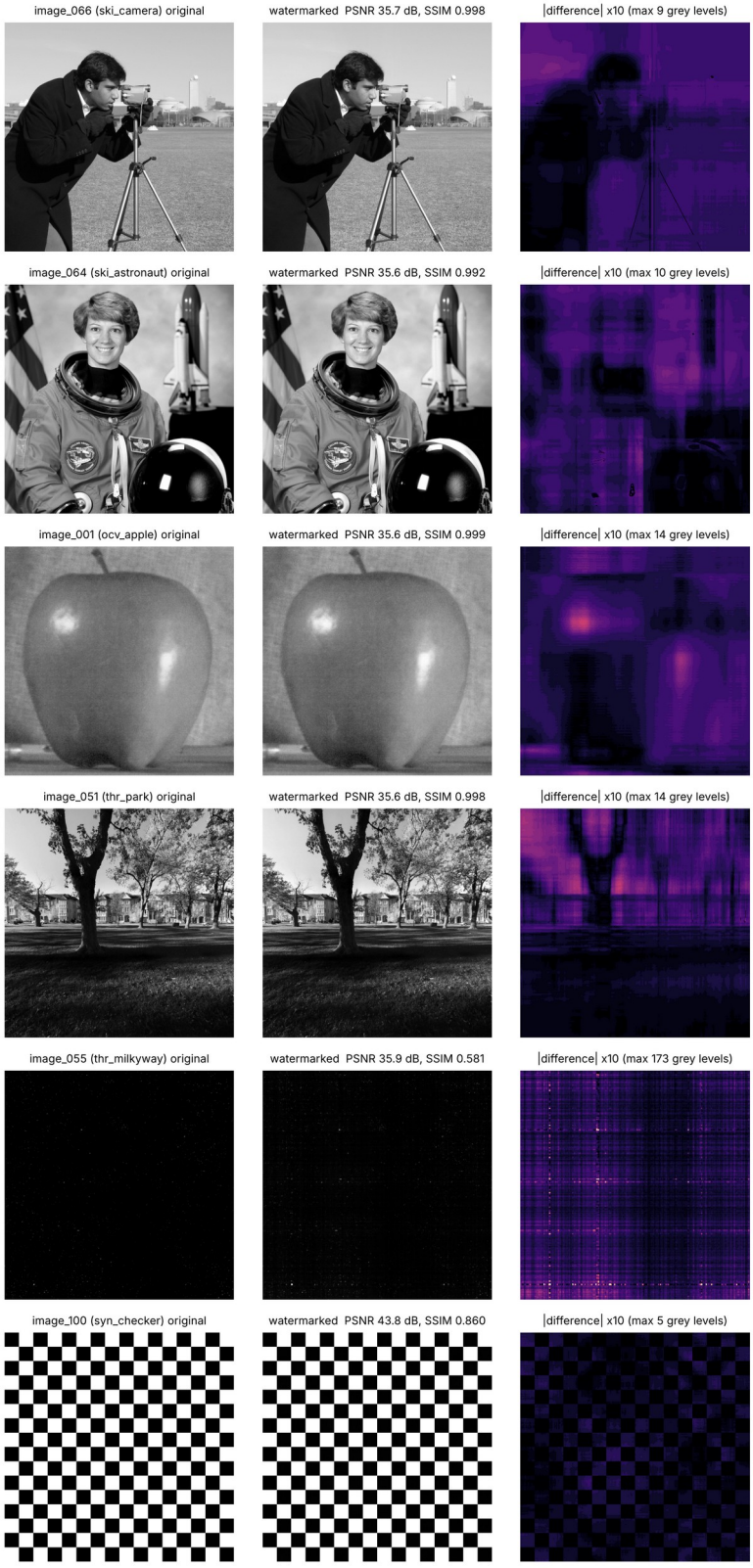



*Figure 8.2: Original, watermarked and amplified (×10) absolute difference for four typical images (image_066, image_064, image_001, image_051) and two failure cases (image_055: near-black sky; image_100: checkerboard).*

**Failure case.** The near-black astronomy image `image_055` has PSNR 35.9 dB but SSIM 0.58: with almost no signal, the added structure and clipping at 0 dominate the local statistics (difference of up to 173 grey levels). PSNR alone would have hidden this.

## 8.2 Robustness under attack (E2)

*Table 8.1: Robustness over 100 images per attack (uint8 protocol). Total: 1277 of 1500 trials ≥ 0.75 (85.1 %); mean NC 0.843.*

| Attack | mean NC [95 % CI] | median NC | NC ≥ 0.75 [Wilson 95 %] | mean ZNCC | mean BER |
|---|---|---|---|---|---|
| No Attack | 0.977 [0.972, 0.980] | 0.981 | 100 % [96, 100] | 0.974 | 0.009 |
| JPEG Q=90 | 0.977 [0.972, 0.979] | 0.981 | 100 % [96, 100] | 0.974 | 0.009 |
| JPEG Q=50 | 0.975 [0.971, 0.979] | 0.981 | 100 % [96, 100] | 0.972 | 0.009 |
| JPEG Q=20 | 0.956 [0.932, 0.975] | 0.980 | 97 % [92, 99] | 0.949 | 0.009 |
| Gaussian Noise (var=0.005) | 0.922 [0.884, 0.952] | 0.969 | 95 % [89, 98] | 0.911 | 0.015 |
| Gaussian Noise (var=0.02) | 0.806 [0.755, 0.852] | 0.902 | 86 % [78, 91] | 0.778 | 0.041 |
| Salt & Pepper (2%) | 0.928 [0.900, 0.950] | 0.955 | 97 % [92, 99] | 0.919 | 0.018 |
| Median Filter 3x3 | 0.967 [0.960, 0.973] | 0.979 | 100 % [96, 100] | 0.964 | 0.008 |
| Gaussian Blur (sigma=1.5) | 0.913 [0.890, 0.932] | 0.942 | 96 % [90, 98] | 0.897 | 0.022 |
| Rotation 5 deg | 0.880 [0.848, 0.907] | 0.932 | 92 % [85, 96] | 0.860 | 0.037 |
| Rotation 15 deg | 0.776 [0.733, 0.818] | 0.866 | 72 % [63, 80] | 0.739 | 0.067 |
| Cropping 25% | 0.022 [0.006, 0.045] | 0.000 | 1 % [0, 5] | -0.036 | 0.140 |
| Scaling 0.5x | 0.938 [0.916, 0.953] | 0.960 | 98 % [93, 99] | 0.926 | 0.017 |
| Histogram Equalization | 0.644 [0.599, 0.688] | 0.719 | 44 % [35, 54] | 0.615 | 0.126 |
| Sharpening | 0.966 [0.954, 0.975] | 0.979 | 99 % [95, 100] | 0.963 | 0.009 |

**Overall,** 1277 of 1500 trials (85.1 %) reach NC ≥ 0.75 when the 100 unattacked reference trials are counted, as the first submission did (25 of 30, 83 %); counting only the 14 attacks it is 1177 of 1400 (84.1 %, mean NC 0.834). With the reference included the mean NC is 0.843 (real images only 0.847, pass rate 85.6 %). Clean extraction gives a mean NC of 0.977 (minimum 0.790, on the checkerboard). By attack family, mean NC / pass rate are: compression 0.969 / 99 %; filtering 0.949 / 98 %; geometric 0.654 / 66 %; global intensity 0.644 / 44 %; noise 0.885 / 93 %; none 0.977 / 100 %.

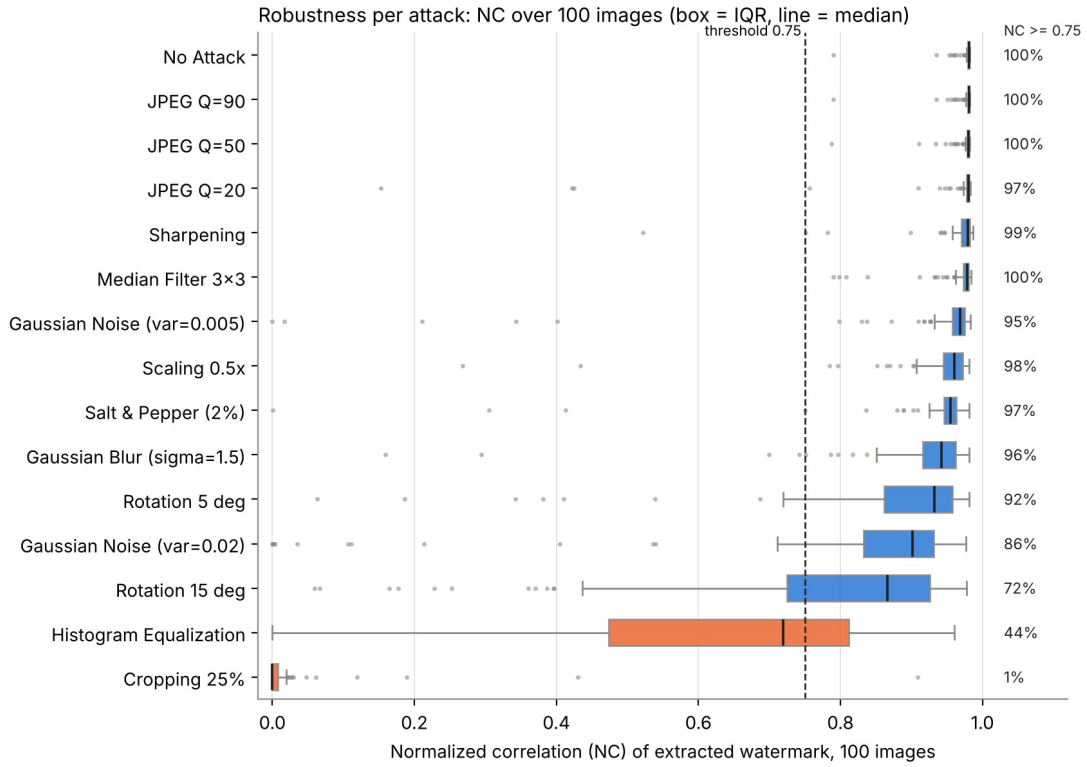



*Figure 8.3: NC per attack over 100 images (box = interquartile range, line = median) and the share of images with NC ≥ 0.75.*

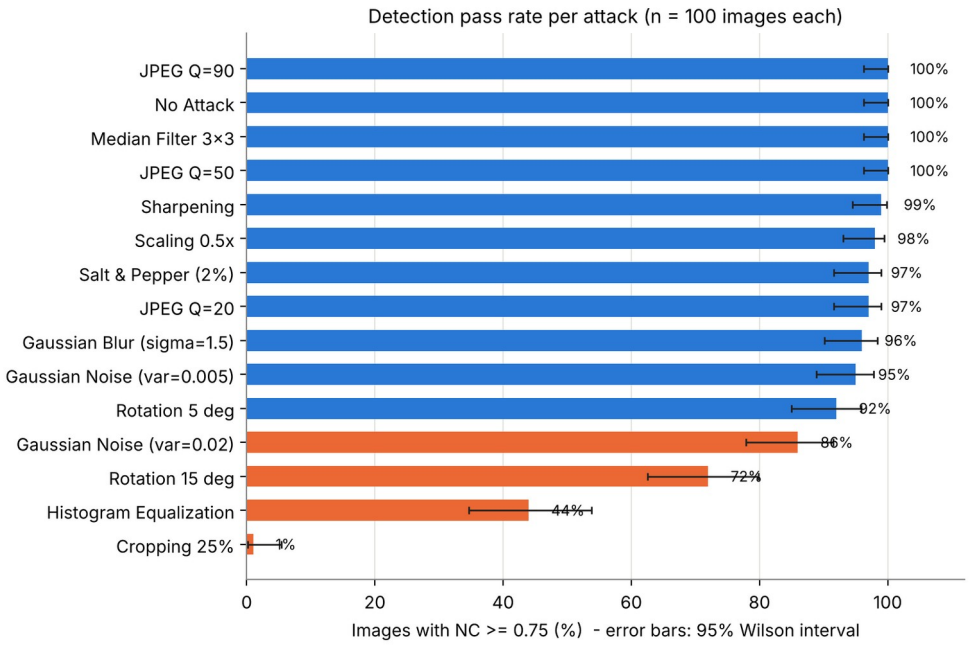



*Figure 8.4: Detection pass rate per attack with 95 % Wilson intervals.*

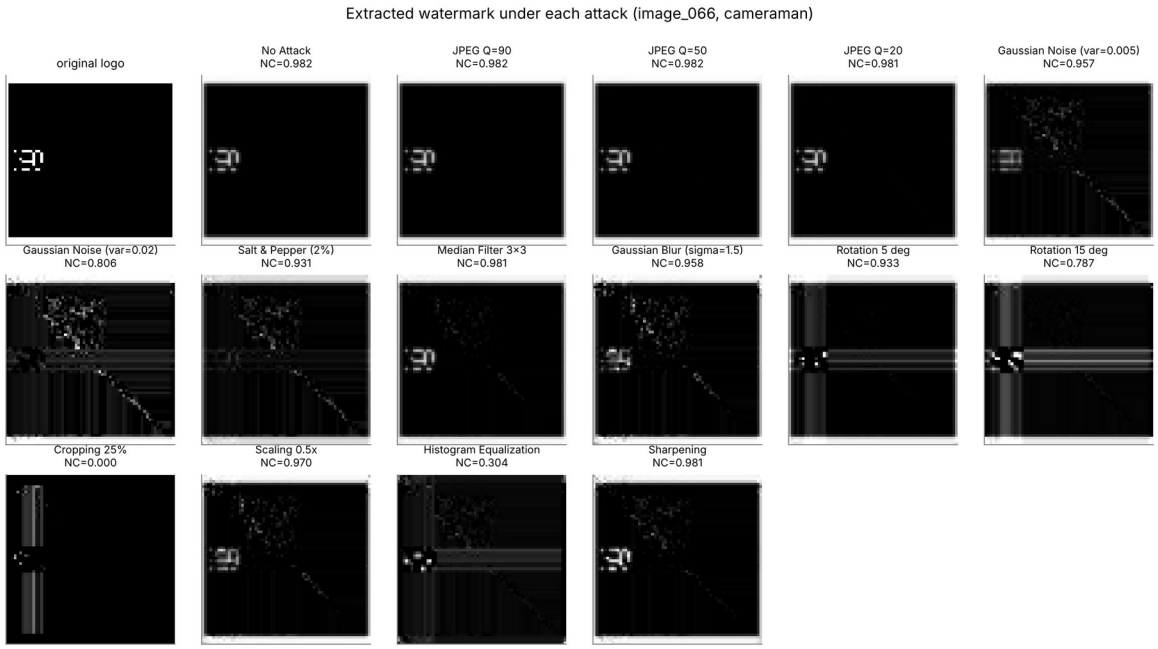



*Figure 8.5: Extracted logo under every attack for one image (image_066, the cameraman).*

**Discussion.** The scheme is very robust to JPEG compression (100 % at Q = 90 and 50, 97 % at Q = 20), median filtering (100 %), sharpening (99 %), scaling (98 %), salt-and-pepper noise (97 %), blur (96 %) and mild Gaussian noise (95 %). It is moderately robust to rotation (92 % at 5°, 72 % at 15°) and strong Gaussian noise (86 %), and it fails under histogram equalisation (44 %) and border cropping (1 %). The first submission's statement that "moderate rotation/scaling were recovered with NC > 0.93 on average" holds for scaling (0.938) but **not** for 5° rotation (0.880) when 100 images are used.

## 8.3 Protocol check

Attacking the 8-bit image (primary) instead of the float image (first submission) changes the mean NC by less than 0.001 for every attack except histogram equalisation (0.644 vs 0.638; up to 0.21 for a

single image); the largest per-image change for the other attacks is 0.009. Mean NC over all trials: 0.8432 (uint8) versus 0.8428 (float).

## 8.4 Which images fail?

Robustness depends on the image. Across the 100 images, NC after most attacks is **negatively** correlated with image contrast (Spearman ρ): Salt & Pepper (2%) -0.68; Gaussian Noise (var=0.02) -0.65; Rotation 15 deg -0.55; Gaussian Noise (var=0.005) -0.44; Rotation 5 deg -0.38; Cropping 25% -0.36; Sharpening -0.34; JPEG Q=20 -0.29. Histogram equalisation correlates with mean brightness instead (ρ = -0.31). These are associations (n = 100, uncorrected p-values), not demonstrated causes.

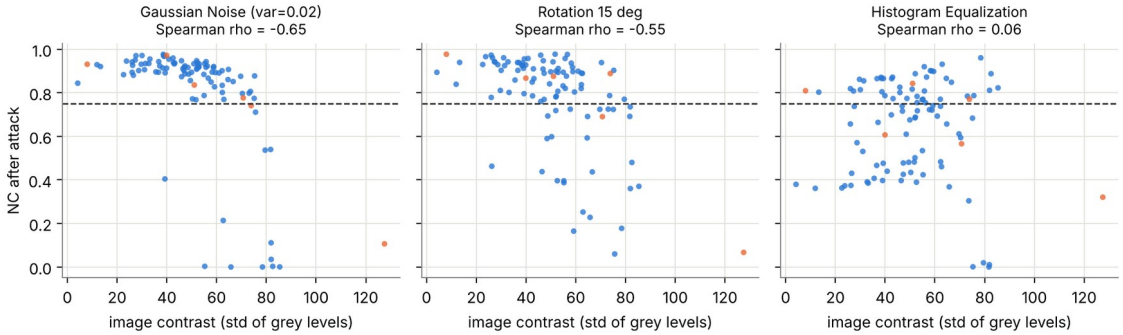



*Figure 8.6: NC after three attacks against image contrast (blue: real, orange: synthetic).*

By category (Appendix C; several categories have only 1–4 images, so these are descriptive): the two document-text images are the least robust to noise (mean NC 0.53) and filtering (0.75), and portraits and documents are the least robust to histogram equalisation (0.40 and 0.39), while natural textures and animals are among the most robust.

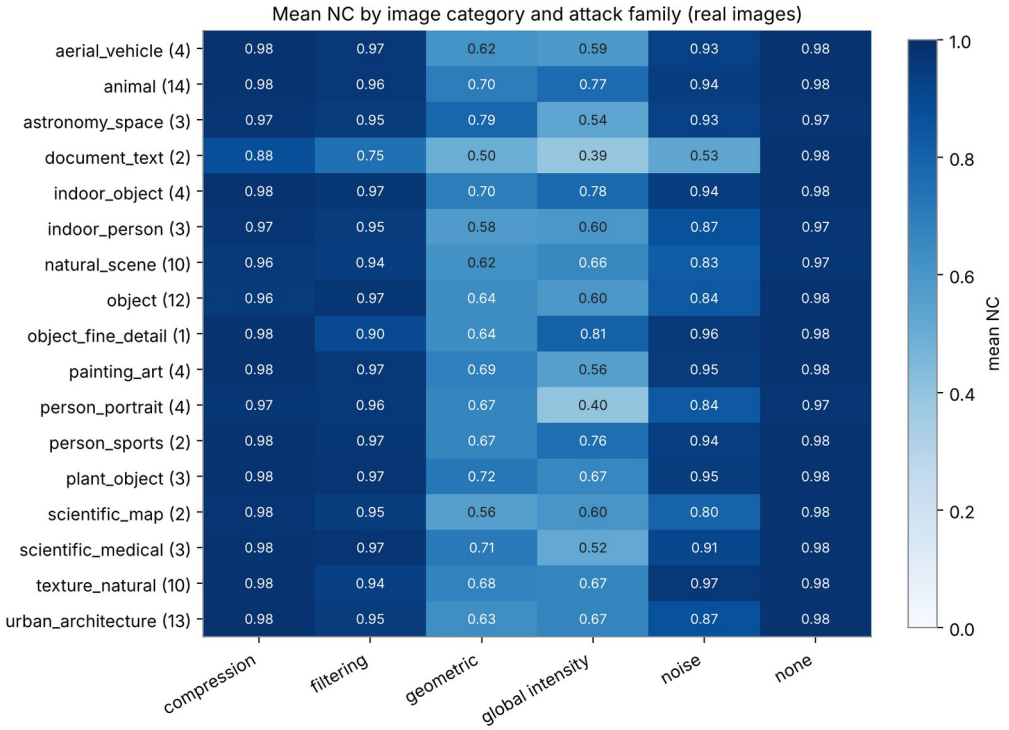



*Figure 8.7: Mean NC by image category and attack family (real images; number of images per category in brackets).*

## 8.5 Embedding strength α (E3)

Mean PSNR falls from 49.1 dB (α = 0.02) to 36.0 dB (0.10) and 24.2 dB (0.40). The effect on robustness depends on the attack. **After JPEG Q=50 the mean NC is essentially flat** (0.960 at α = 0.02, 0.976 at 0.10, 0.973 at 0.40), so the rise shown in the first submission's trade-off figure was an artefact: its NC axis spanned only 0.93–0.99 on a second y-axis, and its JPEG simulation truncated floating-point pixels to 8 bit (a −0.5 grey-level bias), which lowered NC after JPEG at α = 0.02 from 0.981 to 0.929 and had no effect for α ≥ 0.05 (re-creation of the original conditions: `results/exp_jpeg_truncation.txt`). A real trade-off appears under harsher attacks: under Gaussian noise ($\sigma^2 = 0.02$) the pass rate rises from 9 % at α = 0.02 to 58 % at 0.05, 85 % at 0.10, 94 % at 0.20 and 95 % at 0.40, while PSNR drops below 30 dB for α ≥ 0.20 (30.1 dB). The first submission suggested α ≈ 0.15–0.20 "at a small, still-imperceptible PSNR cost"; the measured cost is 32.5 dB and 30.1 dB, i.e. at or just above the conventional 30 dB rule of thumb.

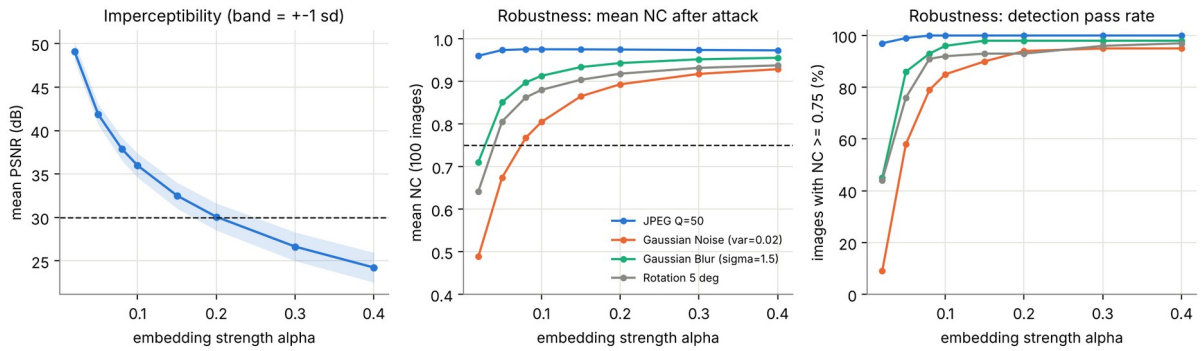



*Figure 8.8: α sweep over 100 images: PSNR (left), mean NC (middle) and pass rate (right) after four attacks; the dashed line in the middle panel marks NC = 0.75.*

## 8.6 False-positive controls (E4)

E4 asks whether a high NC proves the watermark is present. (a) An *unwatermarked other* image decoded with the key of image *i* gives a mean NC of 0.368 (95th percentile 0.804, maximum 0.945); (b) a *watermarked other* image (same logo, wrong key) gives 0.401 (95th percentile 0.832); 9.8 % and 12.6 % of these controls exceed the 0.75 threshold, but only 0.8 % and 1.0 % exceed 0.90. (c) The **same host watermarked with a different logo and decoded with the key of logo A** gives NC(A) = **0.975**, against 0.977 for the genuine pair and 0.977 for the different logo with its own key: the extractor returns the key owner's logo, whatever was embedded (200 trials; ZNCC 0.972). (d) Decoding the image's own *unwatermarked* host with its own key gives NC = 0 by construction, since S_att = S_orig — a degenerate case that must not be read as a positive result.

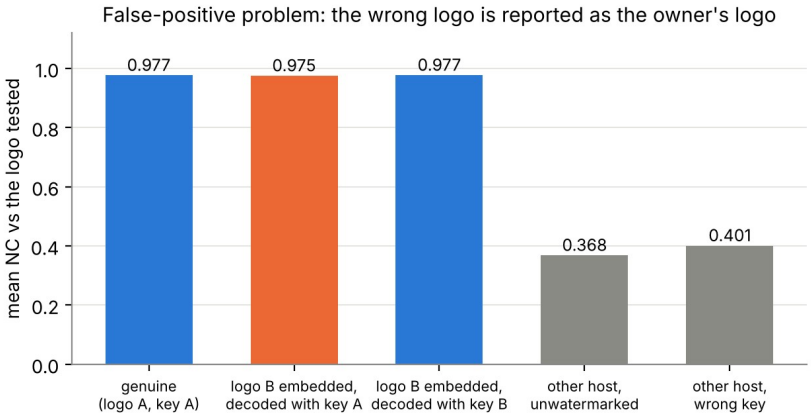



*Figure 8.9: Mean NC for the genuine pair, for a different logo decoded with the owner's key (orange), and for unrelated images.*

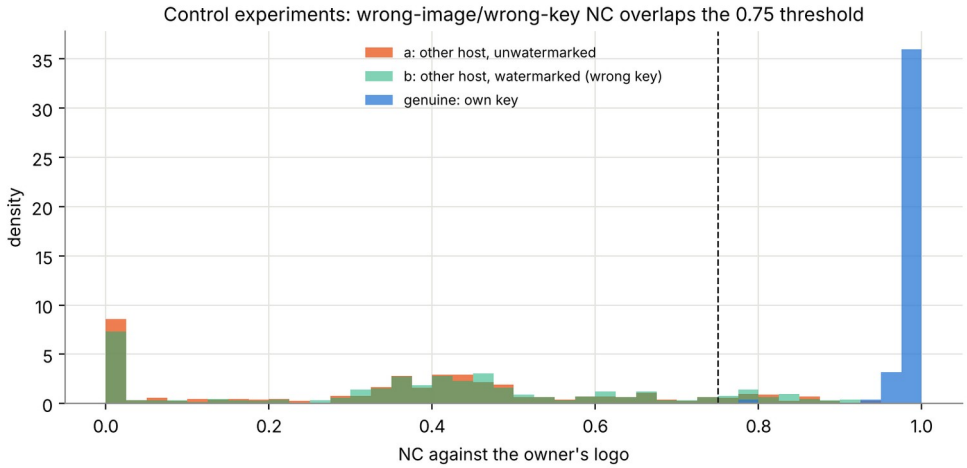



*Figure 8.10: Distribution of NC for genuine extraction and for unrelated-image controls.*

**Interpretation.** NC measures the fidelity of the recovered singular values *given the key*; it does not prove that a particular logo is embedded. Control (c) is the false-positive problem described for SVD schemes in [4][5] (Section 2.2), now measured for this implementation. Wrong-image controls overlap the 0.75 threshold: the 95th percentile of NC for unrelated images is 0.82, and a detector calibrated to a 5 % false-alarm rate would reject 19 % of the attacked genuine trials. A claim of ownership would need an additional safeguard (for example an image-bound signature); that was not implemented here.

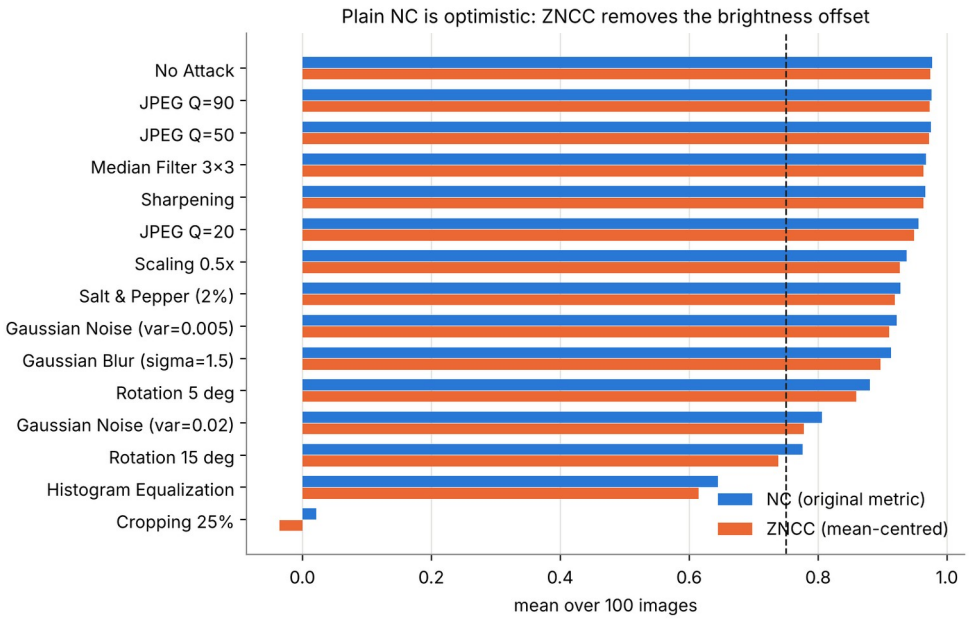



*Figure 8.11: Mean NC and mean ZNCC per attack.*

**NC versus ZNCC.** ZNCC is about 0.02 lower than NC for the clean case (0.974 vs 0.977); over all trials the mean ZNCC is 0.827 and 84.3 % of trials reach 0.75, so the conclusions based on NC are not an artefact of the brightness offset for this logo.

## 8.7 Failure cases

- **Border cropping:** mean NC 0.022; with 43.75 % of the image area zeroed, every singular value changes. The extracted logo is essentially black (BER 0.140 ≈ the white fraction of the logo).
- **Histogram equalisation:** pass rate 44 %; it remaps the whole intensity distribution, and the effect varies strongly between images (first submission: 0.10 for the cameraman, 0.66 for the astronaut).
- **Near-constant images:** SSIM collapses (0.58) though PSNR is 35.9 dB; the checkerboard has the lowest clean NC (0.79).
- **Rotation > 5°:** 72 % pass at 15°; no re-registration is attempted.

## 8.8 Comparison with existing technologies (S1, S2)

**Component level.** Equivalent operations agree numerically (DCT and SVD to ≤ 1.5e-16 relative; Haar DWT to 1e-13; Pillow and OpenCV JPEG bit-identical; 3×3 median identical). OpenCV is much faster for the attack operations (3×3 median ≈ 340×, Gaussian blur 4.4×, rotation 5.1×, down/up-scaling 38×, histogram equalisation 19×; part of this reflects `uint8` versus `float64` input), whereas `scipy.linalg.svd` is only 1.19× faster than NumPy's for the SVD that dominates embedding. Because the whole experiment takes about 80 s, none of this justifies changing libraries.

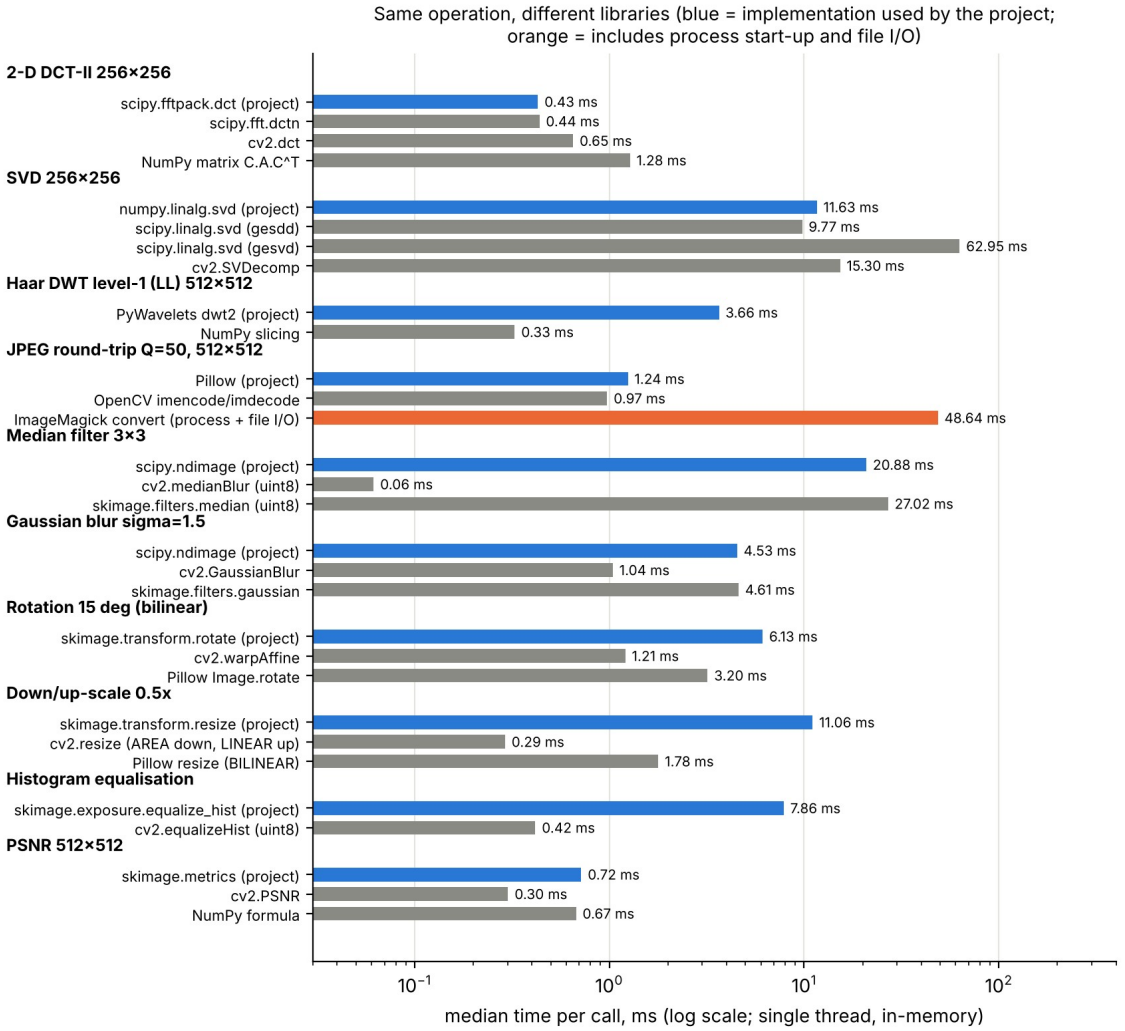



*Figure 8.12: Median time per call for equivalent operations in different libraries (log scale; blue = implementation used by the project; orange includes process start-up and file I/O).*

*Table 8.2: Scheme comparison on 30 real images with the same 4095-bit payload (BER 0.5 = chance).*

| Attack (mean BER, lower is better) | Project | blind-watermark 0.4.4 | 8×8 block DCT | LSB |
|---|---|---|---|---|
| No Attack | 0.010 | 0.004 | 0.000 | 0.000 |
| JPEG Q=50 | 0.010 | 0.004 | 0.296 | 0.426 |
| JPEG Q=20 | 0.010 | 0.074 | 0.469 | 0.531 |
| Gaussian Noise (var=0.02) | 0.038 | 0.005 | 0.182 | 0.500 |
| Median Filter 3x3 | 0.004 | 0.004 | 0.047 | 0.081 |
| Gaussian Blur (sigma=1.5) | 0.022 | 0.004 | 0.106 | 0.456 |
| Rotation 5 deg | 0.029 | 0.214 | 0.493 | 0.504 |
| Cropping 25% | 0.135 | 0.060 | 0.124 | 0.128 |
| Scaling 0.5x | 0.015 | 0.004 | 0.483 | 0.455 |
| **PSNR (dB)** | 35.9 | 42.4 | 35.1 | 69.5 |
| **embed / extract (ms)** | 32 / 17 | 420 / 315 | 26 / 12 | 2 / 1 |

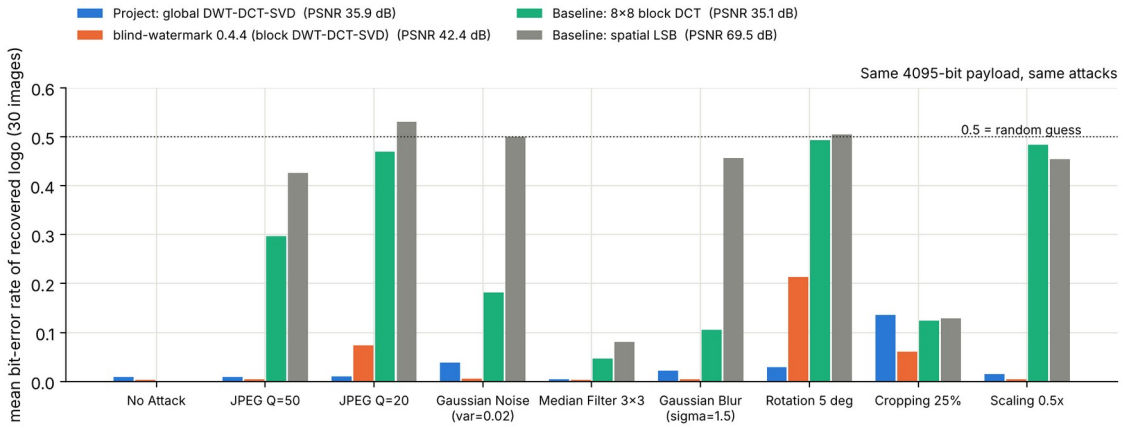



*Figure 8.13: Mean BER of four schemes under nine attacks (PSNR in the legend).*

**Scheme level.** At equal PSNR the project is far more robust than LSB and 8×8 block-DCT embedding, which fall to chance level under JPEG, rescaling and rotation. It is **not superior to the existing blind block-based hybrid** `blind-watermark`: that package achieves a lower BER under JPEG Q=50 (0.004 vs 0.010), noise (0.005 vs 0.038), blur, rescaling and cropping (0.060 vs 0.135), at a higher PSNR (42.4 vs 35.9 dB) and without a key; but it is 13× slower to embed and 18× slower to extract, and it is worse under rotation (0.214 vs 0.029) and JPEG Q=20 (0.074 vs 0.010). Its colour/YUV design and our channel-wise attacks may favour it. What the project contributes is a short, explainable, key-based reference implementation and the evaluation of its weaknesses, not a better algorithm.

## 8.9 Exploratory video test (V1)

*Table 8.3: Per-frame embedding on 60 frames of each of two clips (vtest.avi, Megamind.avi), re-encoded with FFmpeg; means over the two clips.*

| Codec setting | frames decoded | PSNR vs original (dB) | PSNR of codec alone (dB) | mean NC | min NC | file size (kB) |
|---|---|---|---|---|---|---|
| lossless (FFV1) | 120 | 35.6 | ∞ (lossless) | 0.980 | 0.977 | 5289 |
| H.264 CRF 18 | 120 | 35.0 | 44.7 | 0.980 | 0.977 | 381 |
| H.264 CRF 23 | 120 | 34.6 | 41.9 | 0.980 | 0.976 | 181 |
| H.264 CRF 28 | 120 | 33.9 | 39.1 | 0.979 | 0.974 | 88 |
| MJPEG q=5 | 120 | 34.3 | 41.4 | 0.979 | 0.975 | 1244 |
| MJPEG q=15 | 120 | 32.3 | 36.0 | 0.977 | 0.972 | 595 |

All 720 decoded frames (2 clips × 60 frames × 6 settings) yielded NC ≥ 0.972, so lossy video codecs are not the difficulty for this scheme; embedding takes 41 ms per frame (≈ 24 frames per second on one core, 512×512). **Caveats:** this is a feasibility test, not a video-watermarking benchmark — each frame needs its own key (about 2 MB), temporal attacks (frame dropping, re-ordering, frame-rate change) and colour video were not tested, and 60 frames per clip is small. A naive frame-drop test (every tenth frame removed, keys aligned by position) still gave NC 0.973 against 0.980 when aligned by frame ID, presumably because neighbouring frames are almost identical — another symptom of the key-dependence discussed in Section 8.6.

## 8.10 Comparison with the claims of the first submission

*Table 8.4: Claims of the first submission against the 100-image evaluation.*

| First submission | This evaluation (100 images) | Verdict |
|---|---|---|
| mean PSNR 35.6 dB / SSIM 0.996 (2 images) | 36.0 dB / 0.992; 3 images below SSIM 0.98 | confirmed, with exceptions |

| First submission | This evaluation (100 images) | Verdict |
|---|---|---|
| 25/30 trials ≥ 0.75 (83 %) | 1277/1500 (85.1 %) | confirmed |
| JPEG: NC > 0.97 | mean NC 0.977 / 0.975 / 0.956 (Q 90/50/20); pass 100 %/100 %/97 % | holds for Q ≥ 50; Q=20 slightly lower |
| moderate rotation/scaling NC > 0.93 | scaling 0.938; rotation 5°/15° 0.880/0.776 | scaling yes, rotation no |
| cropping NC 0.00; histogram eq. 0.10 / 0.66 | cropping 0.022 (pass 1 %); hist-eq mean 0.644 (pass 44 %) | confirmed; hist-eq varies widely by image |
| robustness rises with α (JPEG Q=50) | flat: 0.960 → 0.973 | not supported (axis + truncation artefact); true for noise/blur/rotation |
| α ≈ 0.15–0.20 at "small" PSNR cost | 32.5 / 30.1 dB | borderline |
| "block DCT", "semi-blind" | global DCT; non-blind | corrected |
| extraction proves ownership (implied) | wrong logo decoded at NC 0.975 | **not supported** |
| hybrids beat single transforms (literature) | not tested; project beats LSB/block-DCT, `blind-watermark` beats project | unverified claim |

# 9. Limitations and Future Scope

## 9.1 Current limitations

- **False-positive problem** (Section 8.6): the extractor returns the key owner's logo; not fixed.
- **Non-blind**: needs a ≈ 2.1 MB key per image holding the original singular values.
- **Geometry:** border cropping and rotation beyond 5° defeat the scheme; no re-synchronisation.
- **Histogram equalisation** and strong noise reduce the pass rate to 44 % and 86 %.
- **Grey-scale only**; clean extraction is not lossless (BER ≈ 0.9 % from logo resampling).
- **Imperceptibility metrics:** PSNR is almost host-independent; SSIM collapses on near-constant images.

## 9.2 Dataset limitations

A convenience sample of 94 real images from GitHub-hosted sample sets (plus six synthetic controls); several categories contain 1–4 images; image rights are mostly not documented upstream; standardisation to grey, centred square crops and 512×512 resampling remove colour, borders and fine detail of the originals. The images are not a recognised benchmark.

## 9.3 Performance, quality and comparison limits

Timings were taken on four shared vCPUs (5–10 % run-to-run variation, no GPU). Only one existing watermarking package was measured, with channel-wise attacks that may favour it. The video test covers 120 distinct frames from two clips.

## 9.4 Future scope

- **Bind the logo to the image** (for example a signature of image features stored in the key) to remove the false-positive problem; verify with the E4 controls.
- **Block-based redundant embedding** with per-block keys to address cropping, and a comparison against the `blind-watermark` design as the new baseline.
- **Geometric re-synchronisation** (log-polar or template-based registration, cf. [22]).
- **Colour:** embed in the luminance channel of a YCbCr image.

- **Video:** shared or index-aware keys, temporal redundancy, tests with frame dropping and re-timing; **real-time or embedded** implementations (≈ 37 ms per $512^2$ frame on one CPU core suggests ≈ 27 frames per second without optimisation).
- **Learned or hybrid approaches** [29][30] as a comparison, if the no-ML constraint is relaxed.

# 10. Conclusion

The project implemented a closed-form hybrid DWT–DCT–SVD watermarking scheme and, in this extended evaluation, tested it on 100 unique images with 14 attacks and an unattacked reference, controls, an α sweep, a technology comparison and a short video test. **Objectives achieved:** the pipeline is implemented, tested (43 tests) and reproducible; imperceptibility is good (mean PSNR 36.0 dB, SSIM 0.992); 85.1 % of attack trials recover a logo with NC ≥ 0.75, with very high robustness to JPEG, median filtering, sharpening and rescaling.

**Main findings:** (1) the first submission's headline numbers are reproducible and largely hold at scale, but moderate rotation and histogram equalisation are weaker than it suggested; (2) the imperceptibility–robustness trade-off in α exists for noise, blur and rotation but not for JPEG, and the earlier figure exaggerated it (zoomed axis, truncation bias); (3) **NC is not proof of ownership** — a different logo decoded with the owner's key is reported at NC 0.98 — and unrelated images exceed the 0.75 threshold in 10–13 % of controls; (4) the scheme is more robust than LSB and block-DCT at equal PSNR but is not better than an existing blind block-based package, which has lower error rates under most attacks but is 13× slower to embed and weaker under rotation; (5) per-frame application to re-encoded video works technically, with per-frame keys.

**Practical significance and lessons.** The scheme is a clear, explainable teaching implementation and a usable robust mark against re-compression and filtering, provided its key is protected and ownership claims are not based on NC alone. The most valuable lesson of the extension is methodological: more images, seeded attacks, controls and honest axes change the conclusions of a two-image study.

# References

[1] I. J. Cox, J. Kilian, F. T. Leighton, and T. Shamoon, "Secure spread spectrum watermarking for multimedia," IEEE Trans. Image Process., vol. 6, no. 12, pp. 1673–1687, Dec. 1997, doi: 10.1109/83.650120.

[2] R. Liu and T. Tan, "An SVD-based watermarking scheme for protecting rightful ownership," IEEE Trans. Multimedia, vol. 4, pp. 121–128, 2002.

[3] E. Ganic and A. M. Eskicioglu, "Robust DWT-SVD domain image watermarking: Embedding data in all frequencies," in Proc. Workshop Multimedia and Security (MM&Sec '04), ACM, 2004, pp. 166–174.

[4] X.-P. Zhang and K. Li, "Comments on 'An SVD-based watermarking scheme for protecting rightful ownership'," IEEE Trans. Multimedia, vol. 7, pp. 593–594, 2005.

[5] G. C.-W. Ting, "Ambiguity attacks on the Ganic–Eskicioglu robust DWT-SVD image watermarking scheme," in Proc. ICISC, 2005, pp. 378–388.

[6] A. Sverdlov, S. Dexter, and A. M. Eskicioglu, "Robust DCT-SVD domain image watermarking for copyright protection: Embedding data in all frequencies," in Proc. EUSIPCO, 2005.

[7] C.-C. Lai and C.-C. Tsai, "Digital image watermarking using discrete wavelet transform and singular value decomposition," IEEE Trans. Instrum. Meas., vol. 59, no. 11, pp. 3060–3063, Nov. 2010.

[8] S. Bekkouch and K. M. Faraoun, "Robust and reversible image watermarking scheme using combined DCT-DWT-SVD transforms," J. Inf. Process. Syst., vol. 11, no. 3, pp. 406–420, 2015, doi: 10.3745/JIPS.02.0021.

[9] M. Begum, J. Ferdush, and M. S. Uddin, "A hybrid robust watermarking system based on discrete cosine transform, discrete wavelet transform, and singular value decomposition," J. King Saud Univ. – Comput. Inf. Sci., vol. 34, no. 8, pp. 5856–5867, 2022.

[10] R. G. van Schyndel, A. Z. Tirkel, and C. F. Osborne, "A digital watermark," in Proc. IEEE Int. Conf. Image Process. (ICIP), vol. 2, 1994, pp. 86–90.

[11] E. Koch and J. Zhao, "Towards robust and hidden image copyright labeling," in Proc. IEEE Workshop Nonlinear Signal Image Process., Neos Marmaras, Greece, Jun. 1995, pp. 452–455.

[12] S. G. Mallat, "A theory for multiresolution signal decomposition: The wavelet representation," IEEE Trans. Pattern Anal. Mach. Intell., vol. 11, no. 7, pp. 674–693, Jul. 1989.

[13] N. Ahmed, T. Natarajan, and K. R. Rao, "Discrete cosine transform," IEEE Trans. Comput., vol. C-23, no. 1, pp. 90–93, Jan. 1974, doi: 10.1109/T-C.1974.223784.

[14] G. K. Wallace, "The JPEG still picture compression standard," IEEE Trans. Consum. Electron., vol. 38, no. 1, pp. xviii–xxxiv, Feb. 1992.

[15] G. H. Golub and C. F. Van Loan, Matrix Computations, 4th ed. Baltimore, MD, USA: Johns Hopkins Univ. Press, 2013.

[16] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, Apr. 2004, doi: 10.1109/TIP.2003.819861.

[17] F. Hartung and M. Kutter, "Multimedia watermarking techniques," Proc. IEEE, vol. 87, no. 7, pp. 1079–1107, Jul. 1999, doi: 10.1109/5.771066.

[18] G. R. Lee et al., "PyWavelets: A Python package for wavelet analysis," J. Open Source Softw., vol. 4, no. 36, p. 1237, 2019, doi: 10.21105/joss.01237.

[19] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[20] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, no. 3, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[21] S. van der Walt et al., "scikit-image: Image processing in Python," PeerJ, vol. 2, p. e453, 2014, doi: 10.7717/peerj.453.

[22] C.-Y. Lin, M. Wu, J. A. Bloom, I. J. Cox, M. L. Miller, and Y. M. Lui, "Rotation, scale, and translation resilient watermarking for images," IEEE Trans. Image Process., vol. 10, no. 5, pp. 767–782, May 2001.

[23] F. Hartung and B. Girod, "Watermarking of uncompressed and compressed video," Signal Process., vol. 66, no. 3, pp. 283–301, 1998.

[24] Pillow (PIL fork), v12.3.0, https://python-pillow.org (licence HPND).

[25] OpenCV, v5.0.0 (`cv2`), https://opencv.org (licence Apache-2.0 from 4.5).

[26] FFmpeg 6.1.1, https://ffmpeg.org.

[27] ImageMagick 6.9.12-98 Q16, https://imagemagick.org.

[28] `blind-watermark` 0.4.4, https://github.com/guofei9987/blind_watermark (PyPI package).

[29] J. Zhu, R. Kaplan, J. Johnson, and L. Fei-Fei, "HiDDeN: Hiding data with deep networks," in Proc. ECCV, 2018, pp. 657–672, doi: 10.1007/978-3-030-01267-0_40.

[30] M. Tancik, B. Mildenhall, and R. Ng, "StegaStamp: Invisible hyperlinks in physical photographs," in Proc. IEEE/CVF CVPR, 2020, pp. 2117–2126, doi: 10.1109/CVPR42600.2020.00219.

[31] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. New York, NY, USA: Pearson, 2018 (cited by the original report without year/pages).

[32] I. J. Cox, M. L. Miller, J. A. Bloom, J. Fridrich, and T. Kalker, Digital Watermarking and Steganography, 2nd ed. Burlington, MA, USA: Morgan Kaufmann, 2008.

*Verification status of each reference (V1 confirmed against the publisher record, V2 confirmed through secondary listings, V3 not independently verified) is given in* `research/REFERENCES.md`*; references that could not be verified were not cited.*

# Appendix A: Installation and execution

```
pip install -r source_code/requirements.txt
cd source_code/src
python build_dataset.py            # data/original + metadata.csv (needs network)
python run_batch_experiment.py     # results/*.csv, outputs/watermarked  (~80 s, 4
cores)
python analysis.py                 # tables, summary, figures
python benchmark_tech.py && python analysis_tech.py   # technology comparison (~2.5 min)
python video_demo.py --frames 60   # exploratory video test (ffmpeg required)
python run_experiment.py           # legacy two-image experiment
cd ../.. && python -m pytest tests -q     # 43 tests
```

# Appendix B: Dataset sources and attribution

*Table B.1: Real-image sources (the repository licence is not a statement about each photograph).*

| Source group | Hosting-repository licence | images |
|---|---|---|
| OpenCV samples | Apache-2.0 | 22 |
| scikit-image data | BSD-3-Clause | 16 |
| three.js examples | MIT | 13 |
| Pillow tests | HPND | 8 |
| Oxford affine-regions (via OpenCV extra) | Apache-2.0 | 7 |
| darknet | YOLO licence (public domain) | 7 |
| OpenCV extra | Apache-2.0 | 6 |
| segment-anything | Apache-2.0 | 3 |
| torchvision gallery | BSD-3-Clause | 2 |
| scipy.datasets | BSD-3-Clause | 2 |
| imageio-binaries | BSD-2-Clause | 2 |
| scikit-learn | BSD-3-Clause | 2 |
| matplotlib | PSF-based (matplotlib licence) | 1 |
| TensorFlow models | Apache-2.0 | 1 |
| mmdetection | Apache-2.0 | 1 |
| transformers test fixtures | Apache-2.0 | 1 |

Upstream-stated licences used in the metadata: CC0 (scikit-image: brick, camera, chelsea, coffee, grass, gravel, retina), NASA public domain (astronaut, Hubble field), CC BY 2.0 (scikit-learn `china.jpg` by *danielbuechele* and `flower.jpg` by *vultilion*; COCO image `hf_cats`, attribution to the Flickr author per COCO terms — not individually verified), Oxford affine-regions images (K. Mikolajczyk *et al.*, research use), Middlebury stereo/flow images (research use). Synthetic controls are generated by `build_dataset.py`. Full per-image attribution: `data/metadata.csv`, `data/README.md`.

# Appendix C: Supplementary results

*Table C.1: Mean NC by category and attack family (all 100 images, uint8 protocol).*

| category | n | compr. | filter. | geom. | glob.int. | noise | clean |
|---|---|---|---|---|---|---|---|
| aerial_vehicle | 4 | 0.98 | 0.97 | 0.62 | 0.59 | 0.93 | 0.98 |
| animal | 14 | 0.98 | 0.96 | 0.70 | 0.77 | 0.94 | 0.98 |
| astronomy_space | 3 | 0.97 | 0.95 | 0.79 | 0.54 | 0.93 | 0.97 |
| document_text | 2 | 0.88 | 0.75 | 0.50 | 0.39 | 0.53 | 0.98 |
| indoor_object | 4 | 0.98 | 0.97 | 0.70 | 0.78 | 0.94 | 0.98 |
| indoor_person | 3 | 0.97 | 0.95 | 0.58 | 0.60 | 0.87 | 0.97 |
| natural_scene | 10 | 0.96 | 0.94 | 0.62 | 0.66 | 0.83 | 0.97 |
| object | 12 | 0.96 | 0.97 | 0.64 | 0.60 | 0.84 | 0.98 |

| category | n | compr. | filter. | geom. | glob.int. | noise | clean |
|---|---|---|---|---|---|---|---|
| object_fine_detail | 1 | 0.98 | 0.90 | 0.64 | 0.81 | 0.96 | 0.98 |
| painting_art | 4 | 0.98 | 0.97 | 0.69 | 0.56 | 0.95 | 0.98 |
| person_portrait | 4 | 0.97 | 0.96 | 0.67 | 0.40 | 0.84 | 0.97 |
| person_sports | 2 | 0.98 | 0.97 | 0.67 | 0.76 | 0.94 | 0.98 |
| plant_object | 3 | 0.98 | 0.97 | 0.72 | 0.67 | 0.95 | 0.98 |
| scientific_map | 2 | 0.98 | 0.95 | 0.56 | 0.60 | 0.80 | 0.98 |
| scientific_medical | 3 | 0.98 | 0.97 | 0.71 | 0.52 | 0.91 | 0.98 |
| synthetic_control | 6 | 0.93 | 0.89 | 0.59 | 0.65 | 0.81 | 0.95 |
| texture_natural | 10 | 0.98 | 0.94 | 0.68 | 0.67 | 0.97 | 0.98 |
| urban_architecture | 13 | 0.98 | 0.95 | 0.63 | 0.67 | 0.87 | 0.98 |

*Table C.2: α sweep (excerpt of results/tables/alpha_sweep_summary.csv).*

| attack | α | PSNR | mean NC | mean ZNCC | mean BER | NC ≥ 0.75 |
|---|---|---|---|---|---|---|
| Gaussian Noise (var=0.02) | 0.02 | 49.1 | 0.489 | 0.377 | 0.160 | 9 % |
| Gaussian Noise (var=0.02) | 0.05 | 41.9 | 0.674 | 0.620 | 0.079 | 58 % |
| Gaussian Noise (var=0.02) | 0.1 | 36.0 | 0.805 | 0.776 | 0.042 | 85 % |
| Gaussian Noise (var=0.02) | 0.2 | 30.1 | 0.893 | 0.878 | 0.029 | 94 % |
| Gaussian Noise (var=0.02) | 0.4 | 24.2 | 0.929 | 0.919 | 0.026 | 95 % |
| Gaussian Blur (sigma=1.5) | 0.02 | 49.1 | 0.711 | 0.648 | 0.088 | 45 % |
| Gaussian Blur (sigma=1.5) | 0.05 | 41.9 | 0.851 | 0.822 | 0.045 | 86 % |
| Gaussian Blur (sigma=1.5) | 0.1 | 36.0 | 0.913 | 0.897 | 0.022 | 96 % |
| Gaussian Blur (sigma=1.5) | 0.2 | 30.1 | 0.943 | 0.933 | 0.016 | 98 % |
| Gaussian Blur (sigma=1.5) | 0.4 | 24.2 | 0.956 | 0.949 | 0.014 | 98 % |
| Rotation 5 deg | 0.02 | 49.1 | 0.642 | 0.570 | 0.112 | 44 % |
| Rotation 5 deg | 0.05 | 41.9 | 0.806 | 0.773 | 0.059 | 76 % |
| Rotation 5 deg | 0.1 | 36.0 | 0.880 | 0.860 | 0.037 | 92 % |
| Rotation 5 deg | 0.2 | 30.1 | 0.918 | 0.904 | 0.029 | 93 % |
| Rotation 5 deg | 0.4 | 24.2 | 0.938 | 0.927 | 0.026 | 97 % |
| JPEG Q=50 | 0.02 | 49.1 | 0.960 | 0.957 | 0.010 | 97 % |
| JPEG Q=50 | 0.05 | 41.9 | 0.974 | 0.970 | 0.008 | 99 % |
| JPEG Q=50 | 0.1 | 36.0 | 0.976 | 0.972 | 0.009 | 100 % |
| JPEG Q=50 | 0.2 | 30.1 | 0.975 | 0.972 | 0.009 | 100 % |
| JPEG Q=50 | 0.4 | 24.2 | 0.973 | 0.970 | 0.011 | 100 % |

*Table C.3: False-positive controls (E4).*

| control | n | mean NC | p95 | max | ≥ 0.75 | mean ZNCC |
|---|---|---|---|---|---|---|
| genuine: own key, clean watermarked image | 100 | 0.977 | 0.981 | 0.983 | 100.0 % | 0.974 |
| a: other host, UNwatermarked, key of image i | 500 | 0.368 | 0.804 | 0.945 | 9.8 % | 0.257 |
| b: other host, watermarked, key of image i | 500 | 0.401 | 0.832 | 0.924 | 12.6 % | 0.298 |
| c: same host, different logo embedded, decoded with key A (NC vs logo | 200 | 0.975 | 0.980 | 0.983 | 100.0 % | 0.972 |
| c-ref: same host, logo B decoded with its own key (NC vs logo B) | 200 | 0.977 | 0.982 | 0.985 | 100.0 % | 0.974 |
| d: own host unwatermarked, own key (degenerate) | 100 | 0.000 | 0.000 | 0.000 | 0.0 % | 0.000 |

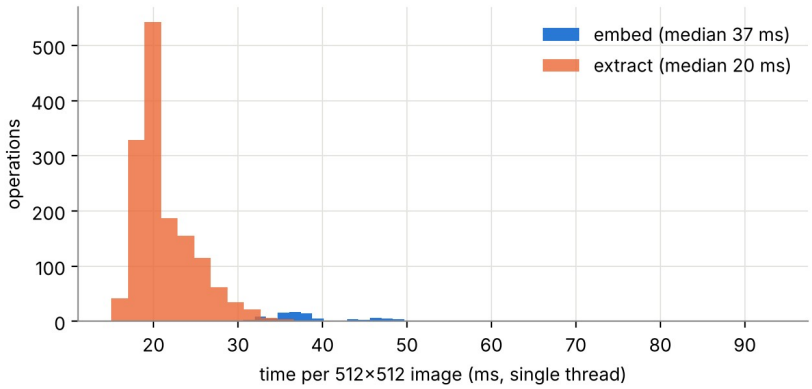



*Figure C.1: Timing of embedding and extraction (single thread, 4 parallel workers).*

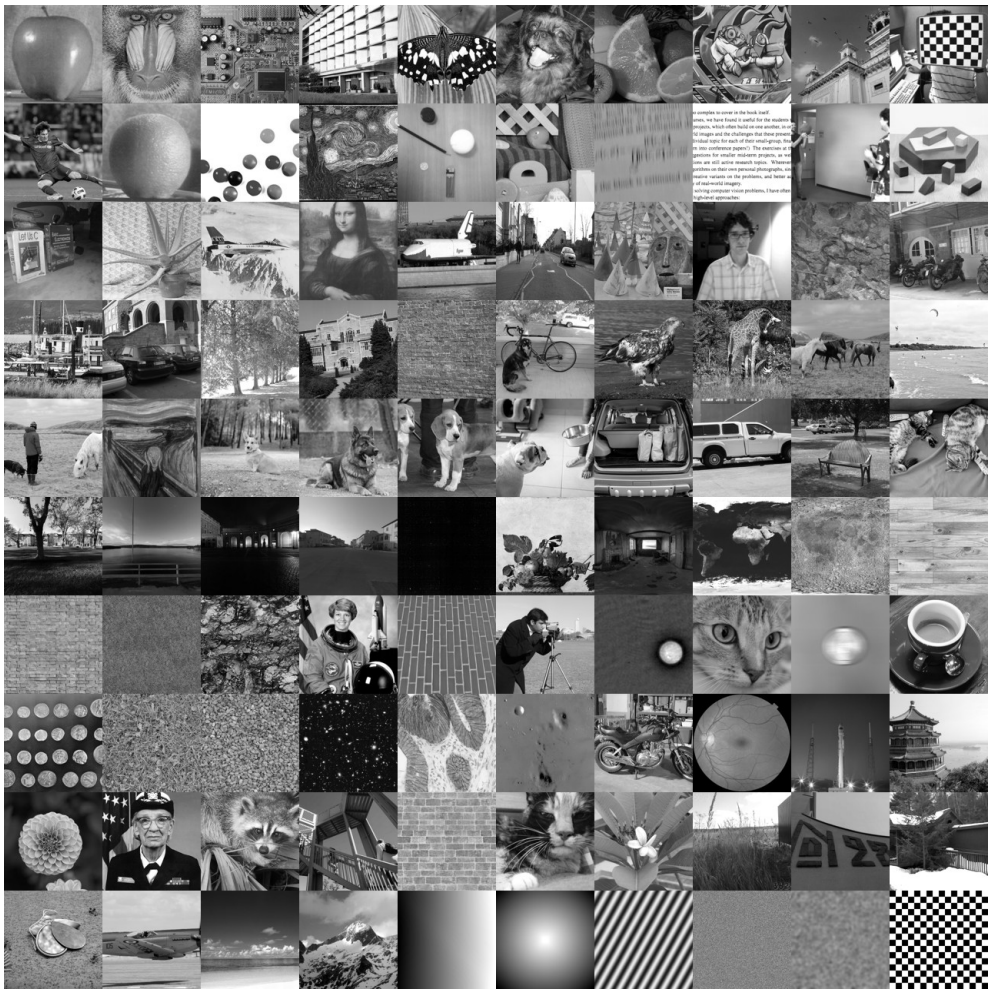


*Figure C.2: Watermarked outputs of all 100 images (same order as Figure 7.1).*

# Appendix D: Test results and configuration

`python -m pytest tests -q`: **43 passed**. Configuration (`source_code/config.json`):

```
{
 "alpha": 0.1,
 "image_size": 512,
 "logo_size": 64,
 "logo_text": "CYBERION",
 "other_logos": [
  "OTHERLOGO",
  "OWNER-B"
```

```
 ],
 "protocols": [
  "uint8",
  "float"
 ],
 "primary_protocol": "uint8",
 "base_seed": 20260713,
 "alphas": [
  0.02,
  0.05,
  0.08,
  0.1,
  0.15,
  0.2,
  0.3,
  0.4
 ],
 "sweep_attacks": [
  "JPEG Q=50",
  "Gaussian Noise (var=0.02)",
  "Gaussian Blur (sigma=1.5)",
  "Rotation 5 deg"
 ],
 "n_controls_per_image": 5,
 "nc_threshold": 0.75
}
```


# Appendix E: Reproduction of the first submission

*Table E.1: Re-execution of the original two-image experiment against Table 5.1 of the first report. Differences are due to seeded noise and JPEG rounding.*

| attack (cameraman) | NC (first report) | NC (re-run) | difference |
|---|---|---|---|
| No Attack | 0.9810 | 0.9810 | +0.0000 |
| JPEG Q=90 | 0.9847 | 0.9818 | -0.0029 |
| JPEG Q=50 | 0.9844 | 0.9817 | -0.0027 |
| JPEG Q=20 | 0.9840 | 0.9811 | -0.0029 |
| Gaussian Noise (var=0.005) | 0.9561 | 0.9601 | +0.0040 |
| Gaussian Noise (var=0.02) | 0.8017 | 0.8017 | +0.0000 |
| Salt & Pepper (2%) | 0.9328 | 0.9327 | -0.0001 |
| Median Filter 3x3 | 0.9806 | 0.9805 | -0.0001 |
| Gaussian Blur (sigma=1.5) | 0.9574 | 0.9572 | -0.0002 |
| Rotation 5 deg | 0.9319 | 0.9318 | -0.0001 |
| Rotation 15 deg | 0.7873 | 0.7879 | +0.0006 |
| Cropping 25% | 0.0000 | 0.0000 | +0.0000 |
| Scaling 0.5x | 0.9691 | 0.9690 | -0.0001 |
| Histogram Equalization | 0.0956 | 0.0950 | -0.0006 |
| Sharpening | 0.9803 | 0.9804 | +0.0001 |

# Appendix F: Corrections to the first report and presentation

1. "Block DCT" → **global** 2-D DCT of the LL band (text, diagrams, deck).
2. "Semi-blind" → **non-blind** (needs S_orig of the unmarked host).
3. Trade-off figure: dual axis with the NC axis zoomed to 0.93–0.99 replaced by separate panels, four attacks and 100 images; the single low point at α = 0.02 was caused by truncating floats before JPEG encoding (`results/exp_jpeg_truncation.txt`).
4. "Cropping 25%" explained as 43.75 % of the image area.

5. "Fifteen attacks": the suite has 14 attacks plus the unattacked reference "No Attack"; pass rates are given with and without the reference.
6. The 0.75 NC threshold, 30 dB and 0.98 SSIM rules of thumb are conventions; no verified literature source was found.
7. "Hybrids repeatedly outperform any single transform" is a reported trend in the literature, not a result of this project.
8. Claim that JPEG alignment of the DCT explains the JPEG robustness: not supported (global DCT, SVD dominates).
9. Reference list rebuilt with verified entries; textbook and documentation entries completed.
10. Added: 100-image evaluation, ZNCC, false-positive controls, technology comparison, video test, unit tests.

# Appendix G: Known limitations of this submission

See Chapter 9 and `documentation/CHANGELOG.md`. Items not completed: no Canva design was edited (no link or connection was provided); no university report template was available; the false-positive problem is measured but not fixed.